

APOSTILA COMPLETA

Tkinter & CustomTkinter

Interfaces Graficas em Python – do basico ao avancado

Guia de referencia com todos os widgets, propriedades e exemplos praticos

Sumario

Sumario	2
Apresentacao	4
Parte I – Fundamentos do Tkinter	5
1. Estrutura de uma aplicacao Tkinter	5
2. Gerenciadores de geometria	5
2.1 pack – empacotamento	5
2.2 grid – grade	6
2.3 place – posicionamento absoluto	6
Parte II – Widgets do Tkinter	8
Tk (Janela Principal)	9
Label (Rotulo)	12
Button (Botao)	14
Entry (Campo de Texto)	16
Text (Area de Texto)	18
Frame (Quadro)	20
Checkbutton (Caixa de Selecao)	22
Radiobutton (Botao de Opcao)	24
Listbox (Lista)	26
Scale (Controle Deslizante)	28
Spinbox (Seletor Numerico)	30
Menu (Menus)	32
Canvas (Tela de Desenho)	34
Parte III – Widgets Tematicos (ttk)	36
ttk.Style (Estilos e Temas)	37
ttk.Combobox (Caixa de Combinacao)	38
ttk.Treeview (Tabela/Arvore)	39
ttk.Notebook (Abas)	40
ttk.Progressbar (Barra de Progresso)	41
Parte IV – Eventos e Vinculacoes (bind)	42

Referencia de eventos comuns	42
Parte V – Caixas de Dialogo e Mensagens	44
messagebox – mensagens	44
filedialog – arquivos	44
colorchooser – cores	45
Parte VI – CustomTkinter	46
CTk e Configuracao Inicial	47
CTkButton (Botao Moderno)	49
CTkLabel (Rotulo Moderno)	51
CTkEntry (Campo Moderno)	53
CTkCheckBox (Caixa Moderna)	55
CTkRadioButton (Opcao Moderna)	57
CTkSwitch (Interruptor)	59
CTkSlider (Controle Deslizante)	61
CTkProgressBar (Barra de Progresso)	63
CTkComboBox e CTkOptionMenu	65
CTkTabview (Abas Modernas)	67
CTkScrollableFrame (Quadro Rolavel)	69
CTkTextbox e CTkSegmentedButton	71
Parte VII – Projetos Completos	73
Calculadora Simples (Tkinter)	74
Lista de Tarefas (To-Do List)	75
Tela de Login Moderna (CustomTkinter)	76
Bloco de Notas com Salvamento (Tkinter)	77
Conversor de Temperatura (Tkinter + ttk)	78
Dashboard com Barra Lateral (CustomTkinter)	79
Parte VIII – Exercicios Propostos	80
Parte IX – Referencias Rapidas	81
Constantes do Tkinter	81
Cursos disponiveis	81
Boas praticas gerais	81
Erros comuns e como evitar	82

Apresentacao

Bem-vindo a Apostila Completa de Tkinter e CustomTkinter. Este material foi elaborado para levar voce do primeiro programa de interface grafica ate a construcao de aplicacoes desktop modernas e profissionais utilizando a linguagem Python.

O Tkinter e a biblioteca padrao de interfaces graficas do Python: ele ja vem instalado com a linguagem e e a porta de entrada ideal para quem deseja criar janelas, botoes, formularios e telas interativas. Ja o CustomTkinter e uma biblioteca de terceiros construida sobre o Tkinter que adiciona uma aparencia moderna, com cantos arredondados, modo claro e escuro, temas de cores e widgets repaginados.

Ao longo desta apostila voce encontrara explicacoes detalhadas de cada widget, tabelas completas de propriedades e metodos, dezenas de exemplos de codigo prontos para executar e projetos completos que integram os conceitos aprendidos. Cada secao inclui ainda boas praticas e os erros mais comuns que voce deve evitar.

Como estudar: leia cada capitulo na ordem, digite e execute os exemplos por conta propria (evite apenas copiar e colar), modifique os valores das propriedades para observar os efeitos e, ao final, resolva os exercicios propostos. A pratica constante e o caminho mais rapido para dominar o desenvolvimento de interfaces graficas.

Pre-requisitos: Conhecimentos basicos de Python (variaveis, funcoes, condicoes, lacos e listas) e o Python 3.8 ou superior instalado.

Instalacao do CustomTkinter: Execute no terminal: `pip install customtkinter`. O Tkinter ja vem incluido no Python.

Parte I – Fundamentos do Tkinter

Nesta primeira parte voce conhecera a estrutura de uma aplicacao Tkinter, criara sua primeira janela e aprendera os tres gerenciadores de geometria que posicionam os widgets na tela. Esses conceitos sao a base de tudo o que vem a seguir.

1. Estrutura de uma aplicacao Tkinter

Toda aplicacao Tkinter segue o mesmo esqueleto: importamos a biblioteca, criamos a janela principal (root), adicionamos widgets, posicionamos esses widgets com um gerenciador de geometria e, por fim, iniciamos o laco principal de eventos com `mainloop()`.

O `mainloop` e um laco infinito que mantem a janela aberta, escuta eventos do usuario (cliques, teclas, redimensionamento) e atualiza a interface. Sem ele, a janela apareceria e fecharia instantaneamente.

Esqueleto basico de toda aplicacao Tkinter

```
import tkinter as tk

# 1. Cria a janela principal
root = tk.Tk()
root.title('Meu Primeiro App')
root.geometry('400x300')

# 2. Cria um widget
label = tk.Label(root, text='Ola, Mundo!', font=('Arial', 16))

# 3. Posiciona o widget
label.pack(pady=40)

# 4. Inicia o laco de eventos
root.mainloop()
```

Memorize esse padrao de quatro passos: criar a janela, criar widgets, posicionar widgets e iniciar o `mainloop`. Praticamente todos os programas desta apostila seguem essa estrutura.

2. Gerenciadores de geometria

Criar um widget nao o faz aparecer na tela: e preciso posicionar-lo com um gerenciador de geometria. O Tkinter oferece tres: `pack`, `grid` e `place`. Nunca misture `pack` e `grid` dentro do mesmo container.

2.1 pack – empacotamento

O `pack` organiza os widgets em blocos, empilhando-os em um dos lados do container (topo por padrao). E o mais simples e rapido para layouts lineares.

Opcoes do pack

Propriedade / Metodo	Tipo	Descricao
<code>side</code>	constante	Lado de ancoragem: TOP, BOTTOM, LEFT, RIGHT.
<code>fill</code>	constante	Preenche o espaco: X, Y, BOTH ou NONE.

Propriedade / Metodo	Tipo	Descricao
expand	bool	Se True, o widget ocupa o espaco extra disponivel.
padx / pady	int	Espacamento externo horizontal/vertical.
ipadx / ipady	int	Espacamento interno adicional.
anchor	constante	Ancoragem dentro do espaco (N, S, E, W, CENTER).

Exemplo: combinando side, fill e expand

```
import tkinter as tk
root = tk.Tk(); root.geometry('300x200')
tk.Label(root, text='Topo', bg='#fca5a5').pack(side='top', fill='x')
tk.Label(root, text='Esquerda', bg='#93c5fd').pack(side='left', fill='y')
tk.Label(root, text='Centro', bg='#86efac').pack(side='left', expand=True, fill='both')
root.mainloop()
```

2.2 grid – grade

O grid organiza os widgets em linhas e colunas, como uma tabela. E o gerenciador ideal para formulários e layouts estruturados.

Opcoes do grid

Propriedade / Metodo	Tipo	Descricao
row / column	int	Linha e coluna onde o widget e colocado.
rowspan / colspan	int	Quantas linhas/colunas o widget ocupa.
sticky	str	Estica o widget na celula: 'n','s','e','w' e combinacoes.
padx / pady	int	Espacamento externo.
rowconfigure(i, weight=)	metodo	Define como a linha cresce ao redimensionar.
columnconfigure(i, weight=)	metodo	Define como a coluna cresce ao redimensionar.

Exemplo: formulario alinhado com grid

```
import tkinter as tk
root = tk.Tk()
tk.Label(root, text='Nome:').grid(row=0, column=0, padx=5, pady=5, sticky='e')
tk.Entry(root).grid(row=0, column=1, padx=5, pady=5)
tk.Label(root, text='Email:').grid(row=1, column=0, padx=5, pady=5, sticky='e')
tk.Entry(root).grid(row=1, column=1, padx=5, pady=5)
tk.Button(root, text='Enviar').grid(row=2, column=0, colspan=2, pady=10)
root.mainloop()
```

2.3 place – posicionamento absoluto

O place posiciona widgets por coordenadas exatas (x, y) ou por proporcao relativa (relx, rely). Oferece controle total, mas e menos flexivel para janelas redimensionaveis.

Opcoes do place

Propriedade / Metodo	Tipo	Descricao
<code>x / y</code>	int	Posicao absoluta em pixels.
<code>relx / rely</code>	float	Posicao relativa (0.0 a 1.0) ao container.
<code>anchor</code>	constante	Ponto de referencia do posicionamento.
<code>relwidth / relheight</code>	float	Tamanho relativo ao container.

Exemplo: centralizando com place

```
import tkinter as tk
root = tk.Tk(); root.geometry('300x200')
tk.Button(root, text='Centro').place(relx=0.5, rely=0.5, anchor='center')
tk.Label(root, text='Canto').place(x=10, y=10)
root.mainloop()
```

Parte II – Widgets do Tkinter

Esta é a parte mais extensa da apostila. Aqui você encontra a referência completa dos widgets clássicos do Tkinter, cada um com sua descrição, tabela de propriedades, exemplos de código e a descrição da tela resultante.

Tk (Janela Principal)

Importacao: import tkinter as tk

A classe Tk representa a janela principal (root) de qualquer aplicacao Tkinter. Toda aplicacao precisa de exatamente uma instancia de Tk, que cria o interpretador Tcl/Tk subjacente e a janela de nivel superior onde os demais widgets sao colocados.

A partir do objeto root configuramos titulo, tamanho, icone, comportamento de redimensionamento e iniciamos o laco principal de eventos com `mainloop()`.

Propriedades e metodos principais

Propriedade / Metodo	Tipo	Descricao
<code>title(texto)</code>	metodo	Define o texto exibido na barra de titulo da janela.
<code>geometry('LxA+X+Y')</code>	metodo	Define largura, altura e posicao. Ex.: '800x600+100+50'.
<code>resizable(x, y)</code>	metodo	Permite (True) ou bloqueia (False) o redimensionamento em cada eixo.
<code>minsize(l, a)</code>	metodo	Define o tamanho minimo da janela.
<code>maxsize(l, a)</code>	metodo	Define o tamanho maximo da janela.
<code>configure(bg=...)</code>	metodo	Altera propriedades como cor de fundo da janela.
<code>iconbitmap(arquivo)</code>	metodo	Define o icone (.ico) da janela no Windows.
<code>protocol(nome, func)</code>	metodo	Associa funcoes a eventos da janela, como 'WM_DELETE_WINDOW'.
<code>attributes('-alpha', v)</code>	metodo	Controla atributos como transparencia, tela cheia ('-fullscreen').
<code>mainloop()</code>	metodo	Inicia o laco de eventos que mantem a janela aberta e responsiva.
<code>destroy()</code>	metodo	Fecha a janela e encerra a aplicacao.
<code>after(ms, func)</code>	metodo	Agenda a execucao de uma funcao apos um intervalo em milissegundos.

Opcoes comuns (compartilhadas com a maioria dos widgets)

Propriedade / Metodo	Tipo	Descricao
<code>background / bg</code>	cor	Cor de fundo do widget. Aceita nomes ('red', 'white') ou hexadecimal ('#2e2e2e').
<code>foreground / fg</code>	cor	Cor do texto (primeiro plano) exibido pelo widget.
<code>font</code>	tupla/str	Fonte do texto. Ex.: ('Arial', 12, 'bold') ou 'Helvetica 10 italic'.
<code>width</code>	int	Largura do widget. Em texto, medida em caracteres; em canvas/imagem, em pixels.
<code>height</code>	int	Altura do widget. Em texto, medida em linhas; em canvas/imagem, em pixels.
<code>relief</code>	constante	Estilo da borda: FLAT, RAISED, SUNKEN, GROOVE, RIDGE, SOLID.
<code>borderwidth / bd</code>	int	Espessura da borda em pixels.

Propriedade / Metodo	Tipo	Descricao
<code>cursor</code>	str	Formato do cursor ao passar sobre o widget. Ex.: 'hand2', 'watch', 'cross'.
<code>padx</code>	int	Espacamento interno horizontal entre conteudo e a borda.
<code>pady</code>	int	Espacamento interno vertical entre conteudo e a borda.
<code>state</code>	constante	Estado do widget: NORMAL, DISABLED ou ACTIVE.
<code>highlightbackground</code>	cor	Cor do retangulo de foco quando o widget NAO tem foco.
<code>highlightcolor</code>	cor	Cor do retangulo de foco quando o widget tem o foco do teclado.
<code>highlightthickness</code>	int	Espessura do retangulo de destaque de foco.
<code>takefocus</code>	bool	Define se o widget recebe foco ao navegar com a tecla Tab.

Exemplos de codigo

Exemplo 1: Janela basica configurada

```
import tkinter as tk

root = tk.Tk()
root.title('Minha Primeira Janela')
root.geometry('640x480+200+100')
root.minsize(400, 300)
root.configure(bg='#1e1e2e')

root.mainloop()
```

Cria a janela principal, define titulo, dimensoes iniciais, tamanho minimo e cor de fundo escura. O mainloop mantem a aplicacao em execucao.

Exemplo 2: Confirmar antes de fechar

```
import tkinter as tk
from tkinter import messagebox

root = tk.Tk()

def ao_fechar():
    if messagebox.askokcancel('Sair', 'Deseja realmente sair?'):
        root.destroy()

root.protocol('WM_DELETE_WINDOW', ao_fechar)
root.mainloop()
```

O metodo protocol intercepta o clique no X da janela e pede confirmacao antes de encerrar o programa.

Exemplo 3: Janela em tela cheia com tecla Esc

```
import tkinter as tk

root = tk.Tk()
root.attributes('-fullscreen', True)

root.bind('<Escape>', lambda e: root.attributes('-fullscreen', False))
root.mainloop()
```

Abre a aplicacao em tela cheia e permite sair desse modo pressionando Esc.

Tela de exemplo: A tela exibe uma janela vazia com a barra de título personalizada e fundo colorido, pronta para receber outros widgets.

Label (Rotulo)

Importacao: import tkinter as tk

O widget Label exibe texto ou imagens estaticas que o usuario nao pode editar. E um dos widgets mais usados para titulos, instrucoes, descricoes e exibicao de resultados dinamicos.

Labels podem ser atualizados em tempo de execucao alterando a opcao text ou vinculando-os a uma variavel de controle (StringVar), o que e ideal para mostrar valores que mudam.

Propriedades e metodos principais

Propriedade / Metodo	Tipo	Descricao
text	str	Texto exibido pelo rotulo.
textvariable	StringVar	Variavel de controle que atualiza o texto automaticamente.
image	PhotoImage	Imagem exibida no lugar ou junto do texto.
compound	constante	Posiciona texto e imagem juntos: LEFT, RIGHT, TOP, BOTTOM, CENTER.
anchor	constante	Alinhamento do conteudo dentro do widget: N, S, E, W, CENTER etc.
justify	constante	Alinhamento de texto com varias linhas: LEFT, RIGHT, CENTER.
wraplength	int	Largura em pixels para quebrar o texto em multiplas linhas.
underline	int	Indice do caractere a ser sublinhado (atalho visual).

Opcoes comuns (compartilhadas com a maioria dos widgets)

Propriedade / Metodo	Tipo	Descricao
background / bg	cor	Cor de fundo do widget. Aceita nomes ('red', 'white') ou hexadecimal ('#2e2e2e').
foreground / fg	cor	Cor do texto (primeiro plano) exibido pelo widget.
font	tupla/str	Fonte do texto. Ex.: ('Arial', 12, 'bold') ou 'Helvetica 10 italic'.
width	int	Largura do widget. Em texto, medida em caracteres; em canvas/imagem, em pixels.
height	int	Altura do widget. Em texto, medida em linhas; em canvas/imagem, em pixels.
relief	constante	Estilo da borda: FLAT, RAISED, SUNKEN, GROOVE, RIDGE, SOLID.
borderwidth / bd	int	Espessura da borda em pixels.
cursor	str	Formato do cursor ao passar sobre o widget. Ex.: 'hand2', 'watch', 'cross'.
padx	int	Espacamento interno horizontal entre conteudo e a borda.
pady	int	Espacamento interno vertical entre conteudo e a borda.
state	constante	Estado do widget: NORMAL, DISABLED ou ACTIVE.
highlightbackground	cor	Cor do retangulo de foco quando o widget NAO tem foco.
highlightcolor	cor	Cor do retangulo de foco quando o widget tem o foco do teclado.

Propriedade / Metodo	Tipo	Descricao
<code>highlightthickness</code>	int	Espessura do retangulo de destaque de foco.
<code>takefocus</code>	bool	Define se o widget recebe foco ao navegar com a tecla Tab.

Exemplos de codigo

Exemplo 1: Label simples

```
import tkinter as tk

root = tk.Tk()
label = tk.Label(root, text='Ola, Tkinter!', font=('Arial', 18, 'bold'),
                 fg='white', bg='#3b82f6', padx=20, pady=10)
label.pack(padx=30, pady=30)
root.mainloop()
```

Cria um rotulo com fonte personalizada, cores e espacamento interno.

Exemplo 2: Label dinamico com StringVar

```
import tkinter as tk

root = tk.Tk()
texto = tk.StringVar(value='Contador: 0')
tk.Label(root, textvariable=texto, font=('Arial', 14)).pack(pady=20)

contador = {'n': 0}
def incrementar():
    contador['n'] += 1
    texto.set(f'Contador: {contador["n"]}')

tk.Button(root, text='Somar', command=incrementar).pack()
root.mainloop()
```

A StringVar conecta o Label ao botao: cada clique atualiza o texto automaticamente.

Exemplo 3: Label com quebra de texto

```
import tkinter as tk

root = tk.Tk()
texto = 'Este e um paragrafo longo que sera quebrado automaticamente '\
        'em varias linhas usando a opcao wraplength.'
tk.Label(root, text=texto, wraplength=200, justify='left').pack(padx=10, pady=10)
root.mainloop()
```

`wraplength` força o texto a quebrar em 200 pixels de largura.

Tela de exemplo: A tela mostra rotulos coloridos, um contador que aumenta a cada clique e um paragrafo quebrado em varias linhas.

Button (Botao)

Importacao: import tkinter as tk

O widget Button cria botoes clicaveis que executam uma funcao (callback) quando acionados. E o principal elemento de interacao em formularios e telas.

A funcao associada e definida na opcao command. Para passar argumentos, usa-se lambda ou functools.partial.

Propriedades e metodos principais

Propriedade / Metodo	Tipo	Descricao
text	str	Rotulo exibido no botao.
command	funcao	Funcao chamada quando o botao e clicado.
image	PhotoImage	Imagem exibida no botao.
compound	constante	Combina texto e imagem (LEFT, TOP etc.).
state	constante	NORMAL, DISABLED ou ACTIVE.
activebackground	cor	Cor de fundo enquanto o botao esta pressionado.
activeforeground	cor	Cor do texto enquanto o botao esta pressionado.
repeatdelay	int	Atraso antes de repetir o comando ao manter pressionado.
default	constante	ACTIVE marca o botao como padrao (acionado por Enter).

Opcoes comuns (compartilhadas com a maioria dos widgets)

Propriedade / Metodo	Tipo	Descricao
background / bg	cor	Cor de fundo do widget. Aceita nomes ('red', 'white') ou hexadecimal ('#2e2e2e').
foreground / fg	cor	Cor do texto (primeiro plano) exibido pelo widget.
font	tupla/str	Fonte do texto. Ex.: ('Arial', 12, 'bold') ou 'Helvetica 10 italic'.
width	int	Largura do widget. Em texto, medida em caracteres; em canvas/imagem, em pixels.
height	int	Altura do widget. Em texto, medida em linhas; em canvas/imagem, em pixels.
relief	constante	Estilo da borda: FLAT, RAISED, SUNKEN, GROOVE, RIDGE, SOLID.
borderwidth / bd	int	Espessura da borda em pixels.
cursor	str	Formato do cursor ao passar sobre o widget. Ex.: 'hand2', 'watch', 'cross'.
padx	int	Espacamento interno horizontal entre conteudo e a borda.
pady	int	Espacamento interno vertical entre conteudo e a borda.
state	constante	Estado do widget: NORMAL, DISABLED ou ACTIVE.
highlightbackground	cor	Cor do retangulo de foco quando o widget NAO tem foco.
highlightcolor	cor	Cor do retangulo de foco quando o widget tem o foco do teclado.

Propriedade / Metodo	Tipo	Descricao
<code>highlightthickness</code>	int	Espessura do retangulo de destaque de foco.
<code>takefocus</code>	bool	Define se o widget recebe foco ao navegar com a tecla Tab.

Exemplos de codigo

Exemplo 1: Botao com callback

```
import tkinter as tk

root = tk.Tk()
def saudar():
    print('Botao clicado!')

tk.Button(root, text='Clique aqui', command=saudar,
          bg='#22c55e', fg='white', font=('Arial', 12),
          activebackground='#16a34a').pack(padx=40, pady=40)
root.mainloop()
```

O parametro command recebe a funcao saudar (sem parenteses) que e executada no clique.

Exemplo 2: Passando argumentos com lambda

```
import tkinter as tk

root = tk.Tk()
def mostrar(valor):
    print('Voce escolheu:', valor)

for cor in ['Vermelho', 'Verde', 'Azul']:
    tk.Button(root, text=cor,
              command=lambda c=cor: mostrar(c)).pack(fill='x', padx=10, pady=2)
root.mainloop()
```

O lambda com c=cor captura o valor correto de cada iteracao, evitando o erro classico de closures em loops.

Exemplo 3: Habilitar e desabilitar botao

```
import tkinter as tk

root = tk.Tk()
btn = tk.Button(root, text='Acao', state='disabled')
btn.pack(pady=10)

def alternar():
    novo = 'normal' if btn['state'] == 'disabled' else 'disabled'
    btn.config(state=novo)

tk.Button(root, text='Alternar', command=alternar).pack()
root.mainloop()
```

Mostra como ler e alterar o estado de um botao em tempo de execucao.

Tela de exemplo: A tela apresenta botoes coloridos, uma lista de botoes de cores e um par de botoes que se habilitam/desabilitam mutuamente.

Entry (Campo de Texto)

Importacao: import tkinter as tk

O widget Entry recebe entrada de texto de uma unica linha do usuario, ideal para nomes, e-mails, senhas e numeros.

O conteudo e lido com get() ou vinculado a uma StringVar. A opcao show mascara caracteres para campos de senha.

Propriedades e metodos principais

Propriedade / Metodo	Tipo	Descricao
<code>textvariable</code>	StringVar	Variavel ligada ao conteudo do campo.
<code>show</code>	str	Caractere de mascara (ex.: '*' para senhas).
<code>justify</code>	constante	Alinhamento do texto digitado.
<code>state</code>	constante	NORMAL, DISABLED ou 'readonly'.
<code>validate</code>	constante	Quando validar: 'key', 'focus', 'all' etc.
<code>validatecommand</code>	tupla	Funcao registrada que valida a entrada.
<code>insert(indice, txt)</code>	metodo	Insere texto na posicao indicada.
<code>delete(ini, fim)</code>	metodo	Remove caracteres do intervalo.
<code>get()</code>	metodo	Retorna o texto atual do campo.

Opcoes comuns (compartilhadas com a maioria dos widgets)

Propriedade / Metodo	Tipo	Descricao
<code>background / bg</code>	cor	Cor de fundo do widget. Aceita nomes ('red', 'white') ou hexadecimal ('#2e2e2e').
<code>foreground / fg</code>	cor	Cor do texto (primeiro plano) exibido pelo widget.
<code>font</code>	tupla/str	Fonte do texto. Ex.: ('Arial', 12, 'bold') ou 'Helvetica 10 italic'.
<code>width</code>	int	Largura do widget. Em texto, medida em caracteres; em canvas/imagem, em pixels.
<code>height</code>	int	Altura do widget. Em texto, medida em linhas; em canvas/imagem, em pixels.
<code>relief</code>	constante	Estilo da borda: FLAT, RAISED, SUNKEN, GROOVE, RIDGE, SOLID.
<code>borderwidth / bd</code>	int	Espessura da borda em pixels.
<code>cursor</code>	str	Formato do cursor ao passar sobre o widget. Ex.: 'hand2', 'watch', 'cross'.
<code>padx</code>	int	Espacamento interno horizontal entre conteudo e a borda.
<code>pady</code>	int	Espacamento interno vertical entre conteudo e a borda.
<code>state</code>	constante	Estado do widget: NORMAL, DISABLED ou ACTIVE.
<code>highlightbackground</code>	cor	Cor do retangulo de foco quando o widget NAO tem foco.
<code>highlightcolor</code>	cor	Cor do retangulo de foco quando o widget tem o foco do teclado.

Propriedade / Metodo	Tipo	Descricao
<code>highlightthickness</code>	int	Espessura do retangulo de destaque de foco.
<code>takefocus</code>	bool	Define se o widget recebe foco ao navegar com a tecla Tab.

Exemplos de codigo

Exemplo 1: Campo de nome e leitura

```
import tkinter as tk

root = tk.Tk()
tk.Label(root, text='Nome:').pack()
campo = tk.Entry(root, width=30, font=('Arial', 12))
campo.pack(padx=10, pady=5)

def ler():
    print('Nome digitado:', campo.get())

tk.Button(root, text='Enviar', command=ler).pack(pady=5)
root.mainloop()
```

O metodo `get()` retorna o texto digitado quando o botao e pressionado.

Exemplo 2: Campo de senha

```
import tkinter as tk

root = tk.Tk()
tk.Label(root, text='Senha:').pack()
senha = tk.Entry(root, show='*', width=25)
senha.pack(padx=10, pady=5)
root.mainloop()
```

A opcao `show='*'` oculta os caracteres digitados para proteger a senha.

Exemplo 3: Validacao para aceitar apenas numeros

```
import tkinter as tk

root = tk.Tk()
def so_numeros(novo):
    return novo.isdigit() or novo == ''

vcmd = (root.register(so_numeros), '%P')
tk.Entry(root, validate='key', validatecommand=vcmd).pack(padx=10, pady=10)
root.mainloop()
```

register cria um comando de validacao; '%P' passa o valor que o campo teria se a edicao fosse aceita.

Tela de exemplo: A tela exibe um formulario com campo de nome, campo de senha mascarado e um campo que aceita somente digitos.

Text (Area de Texto)

Importacao: import tkinter as tk

O widget Text e uma area de edicao de texto com multiplas linhas, suportando formatacao por tags, insercao de imagens e janelas embutidas.

Os indices seguem o formato 'linha.coluna' (ex.: '1.0' para o inicio). E muito usado em editores, logs e campos de comentarios.

Propriedades e metodos principais

Propriedade / Metodo	Tipo	Descricao
<code>wrap</code>	constante	Modo de quebra: WORD, CHAR ou NONE.
<code>insert(indice, txt)</code>	metodo	Inserir texto na posicao informada (ex.: 'end').
<code>get(ini, fim)</code>	metodo	Retorna o texto entre dois indices.
<code>delete(ini, fim)</code>	metodo	Remove o texto do intervalo.
<code>tag_add(nome, ini, fim)</code>	metodo	Marca um trecho com uma tag.
<code>tag_config(nome, ...)</code>	metodo	Define a aparencia de uma tag (cor, fonte).
<code>see(indice)</code>	metodo	Rola a area para tornar o indice visivel.
<code>index('end')</code>	metodo	Retorna o indice do final do conteudo.

Opcoes comuns (compartilhadas com a maioria dos widgets)

Propriedade / Metodo	Tipo	Descricao
<code>background / bg</code>	cor	Cor de fundo do widget. Aceita nomes ('red', 'white') ou hexadecimal ('#2e2e2e').
<code>foreground / fg</code>	cor	Cor do texto (primeiro plano) exibido pelo widget.
<code>font</code>	tupla/str	Fonte do texto. Ex.: ('Arial', 12, 'bold') ou 'Helvetica 10 italic'.
<code>width</code>	int	Largura do widget. Em texto, medida em caracteres; em canvas/imagem, em pixels.
<code>height</code>	int	Altura do widget. Em texto, medida em linhas; em canvas/imagem, em pixels.
<code>relief</code>	constante	Estilo da borda: FLAT, RAISED, SUNKEN, GROOVE, RIDGE, SOLID.
<code>borderwidth / bd</code>	int	Espessura da borda em pixels.
<code>cursor</code>	str	Formato do cursor ao passar sobre o widget. Ex.: 'hand2', 'watch', 'cross'.
<code>padx</code>	int	Espacamento interno horizontal entre conteudo e a borda.
<code>pady</code>	int	Espacamento interno vertical entre conteudo e a borda.
<code>state</code>	constante	Estado do widget: NORMAL, DISABLED ou ACTIVE.
<code>highlightbackground</code>	cor	Cor do retangulo de foco quando o widget NAO tem foco.
<code>highlightcolor</code>	cor	Cor do retangulo de foco quando o widget tem o foco do teclado.
<code>highlightthickness</code>	int	Espessura do retangulo de destaque de foco.

Propriedade / Metodo	Tipo	Descricao
<code>takefocus</code>	bool	Define se o widget recebe foco ao navegar com a tecla Tab.

Exemplos de código

Exemplo 1: Area de texto com rolagem

```
import tkinter as tk

root = tk.Tk()
area = tk.Text(root, width=40, height=10, wrap='word')
scroll = tk.Scrollbar(root, command=area.yview)
area.config(yscrollcommand=scroll.set)
area.pack(side='left', fill='both', expand=True)
scroll.pack(side='right', fill='y')
root.mainloop()
```

A barra de rolagem é ligada ao Text por yview/yscrollcommand.

Exemplo 2: Formatacao com tags

```
import tkinter as tk

root = tk.Tk()
t = tk.Text(root, width=40, height=6)
t.pack(padx=10, pady=10)
t.insert('end', 'Texto normal e ')
t.insert('end', 'texto destacado', 'destaque')
t.tag_config('destaque', foreground='red', font=('Arial', 12, 'bold'))
root.mainloop()
```

Tags permitem aplicar cores e fontes diferentes a trechos específicos.

Exemplo 3: Ler todo o conteudo

```
import tkinter as tk

root = tk.Tk()
t = tk.Text(root, height=5)
t.pack()
def salvar():
    conteudo = t.get('1.0', 'end-1c')
    print(conteudo)

tk.Button(root, text='Salvar', command=salvar).pack()
root.mainloop()
```

'1.0' é o início e 'end-1c' evita capturar a quebra de linha final extra.

Tela de exemplo: A tela mostra um editor com rolagem, trechos coloridos por tags e um botão que imprime todo o conteúdo no console.

Frame (Quadro)

Importacao: import tkinter as tk

O Frame e um container retangular usado para agrupar e organizar outros widgets. E essencial para criar layouts complexos dividindo a janela em regioes.

Frames podem ter cor e borda proprias e servem como base para subdividir telas em cabecalho, conteudo e rodape.

Propriedades e metodos principais

Propriedade / Metodo	Tipo	Descricao
bg	cor	Cor de fundo do quadro.
width / height	int	Dimensoes em pixels (requer propagate(False)).
relief	constante	Estilo da borda.
bd	int	Espessura da borda.
pack_propagate(bool)	metodo	Controla se o frame se ajusta ao conteudo.

Opcoes comuns (compartilhadas com a maioria dos widgets)

Propriedade / Metodo	Tipo	Descricao
background / bg	cor	Cor de fundo do widget. Aceita nomes ('red', 'white') ou hexadecimal ('#2e2e2e').
foreground / fg	cor	Cor do texto (primeiro plano) exibido pelo widget.
font	tupla/str	Fonte do texto. Ex.: ('Arial', 12, 'bold') ou 'Helvetica 10 italic'.
width	int	Largura do widget. Em texto, medida em caracteres; em canvas/imagem, em pixels.
height	int	Altura do widget. Em texto, medida em linhas; em canvas/imagem, em pixels.
relief	constante	Estilo da borda: FLAT, RAISED, SUNKEN, GROOVE, RIDGE, SOLID.
borderwidth / bd	int	Espessura da borda em pixels.
cursor	str	Formato do cursor ao passar sobre o widget. Ex.: 'hand2', 'watch', 'cross'.
padx	int	Espacamento interno horizontal entre conteudo e a borda.
pady	int	Espacamento interno vertical entre conteudo e a borda.
state	constante	Estado do widget: NORMAL, DISABLED ou ACTIVE.
highlightbackground	cor	Cor do retangulo de foco quando o widget NAO tem foco.
highlightcolor	cor	Cor do retangulo de foco quando o widget tem o foco do teclado.
highlightthickness	int	Espessura do retangulo de destaque de foco.
takefocus	bool	Define se o widget recebe foco ao navegar com a tecla Tab.

Exemplos de codigo

Exemplo 1: Dividindo a janela em regioes

```
import tkinter as tk

root = tk.Tk(); root.geometry('400x300')
topo = tk.Frame(root, bg='#3b82f6', height=50)
topo.pack(fill='x')
corpo = tk.Frame(root, bg='#e5e7eb')
corpo.pack(fill='both', expand=True)
rodape = tk.Frame(root, bg='#1f2937', height=30)
rodape.pack(fill='x')
root.mainloop()
```

Tres frames criam cabecalho, corpo expansivel e rodape.

Exemplo 2: Frame com borda

```
import tkinter as tk

root = tk.Tk()
card = tk.Frame(root, bd=2, relief='groove', padx=20, pady=20)
card.pack(padx=20, pady=20)
tk.Label(card, text='Conteudo dentro do card').pack()
root.mainloop()
```

Um frame com relevo agrupa visualmente o conteudo como um cartao.

Tela de exemplo: A tela mostra a janela dividida em cabecalho azul, corpo cinza e rodape escuro, alem de um cartao com borda em relevo.

Checkbox (Caixa de Selecao)

Importacao: import tkinter as tk

O Checkbutton e uma caixa que pode ser marcada ou desmarcada, representando opcoes booleanas independentes.

Seu estado e armazenado em uma variavel de controle (IntVar ou BooleanVar) indicada pela opcao variable.

Propriedades e metodos principais

Propriedade / Metodo	Tipo	Descricao
text	str	Texto ao lado da caixa.
variable	IntVar/BooleanVar	Variavel que guarda o estado (marcado/desmarcado).
onvalue	valor	Valor quando marcado.
offvalue	valor	Valor quando desmarcado.
command	funcao	Funcao chamada ao alternar o estado.

Opcoes comuns (compartilhadas com a maioria dos widgets)

Propriedade / Metodo	Tipo	Descricao
background / bg	cor	Cor de fundo do widget. Aceita nomes ('red', 'white') ou hexadecimal ('#2e2e2e').
foreground / fg	cor	Cor do texto (primeiro plano) exibido pelo widget.
font	tupla/str	Fonte do texto. Ex.: ('Arial', 12, 'bold') ou 'Helvetica 10 italic'.
width	int	Largura do widget. Em texto, medida em caracteres; em canvas/imagem, em pixels.
height	int	Altura do widget. Em texto, medida em linhas; em canvas/imagem, em pixels.
relief	constante	Estilo da borda: FLAT, RAISED, SUNKEN, GROOVE, RIDGE, SOLID.
borderwidth / bd	int	Espessura da borda em pixels.
cursor	str	Formato do cursor ao passar sobre o widget. Ex.: 'hand2', 'watch', 'cross'.
padx	int	Espacamento interno horizontal entre conteudo e a borda.
pady	int	Espacamento interno vertical entre conteudo e a borda.
state	constante	Estado do widget: NORMAL, DISABLED ou ACTIVE.
highlightbackground	cor	Cor do retangulo de foco quando o widget NAO tem foco.
highlightcolor	cor	Cor do retangulo de foco quando o widget tem o foco do teclado.
highlightthickness	int	Espessura do retangulo de destaque de foco.
takefocus	bool	Define se o widget recebe foco ao navegar com a tecla Tab.

Exemplos de código

Exemplo 1: Caixa de seleção simples

```
import tkinter as tk

root = tk.Tk()
aceito = tk.BooleanVar()
tk.Checkbutton(root, text='Aceito os termos', variable=aceito).pack(pady=10)
def verificar():
    print('Aceitou?', aceito.get())
tk.Button(root, text='OK', command=verificar).pack()
root.mainloop()
```

A BooleanVar retorna True/False conforme o estado da caixa.

Exemplo 2: Varias opções

```
import tkinter as tk

root = tk.Tk()
opcoes = {}
for item in ['Email', 'SMS', 'Push']:
    v = tk.BooleanVar()
    tk.Checkbutton(root, text=item, variable=v).pack(anchor='w')
    opcoes[item] = v
tk.Button(root, text='Conferir',
          command=lambda: print({k: v.get() for k, v in opcoes.items()})).pack()
root.mainloop()
```

Um dicionário guarda uma variável por opção, permitindo ler todas de uma vez.

Tela de exemplo: A tela mostra caixas de seleção de termos e preferências de notificação.

Radiobutton (Botao de Opcao)

Importacao: import tkinter as tk

Radiobuttons permitem escolher uma unica opcao entre varias mutuamente exclusivas. Botoes do mesmo grupo compartilham a mesma variavel de controle.

Cada botao define um value diferente; a variavel guarda o value do botao selecionado.

Propriedades e metodos principais

Propriedade / Metodo	Tipo	Descricao
<code>text</code>	str	Texto da opcao.
<code>variable</code>	Var	Variavel compartilhada pelo grupo.
<code>value</code>	valor	Valor atribuido quando este botao e selecionado.
<code>command</code>	funcao	Funcao chamada ao selecionar.

Opcoes comuns (compartilhadas com a maioria dos widgets)

Propriedade / Metodo	Tipo	Descricao
<code>background / bg</code>	cor	Cor de fundo do widget. Aceita nomes ('red', 'white') ou hexadecimal ('#2e2e2e').
<code>foreground / fg</code>	cor	Cor do texto (primeiro plano) exibido pelo widget.
<code>font</code>	tupla/str	Fonte do texto. Ex.: ('Arial', 12, 'bold') ou 'Helvetica 10 italic'.
<code>width</code>	int	Largura do widget. Em texto, medida em caracteres; em canvas/imagem, em pixels.
<code>height</code>	int	Altura do widget. Em texto, medida em linhas; em canvas/imagem, em pixels.
<code>relief</code>	constante	Estilo da borda: FLAT, RAISED, SUNKEN, GROOVE, RIDGE, SOLID.
<code>borderwidth / bd</code>	int	Espessura da borda em pixels.
<code>cursor</code>	str	Formato do cursor ao passar sobre o widget. Ex.: 'hand2', 'watch', 'cross'.
<code>padx</code>	int	Espacamento interno horizontal entre conteudo e a borda.
<code>pady</code>	int	Espacamento interno vertical entre conteudo e a borda.
<code>state</code>	constante	Estado do widget: NORMAL, DISABLED ou ACTIVE.
<code>highlightbackground</code>	cor	Cor do retangulo de foco quando o widget NAO tem foco.
<code>highlightcolor</code>	cor	Cor do retangulo de foco quando o widget tem o foco do teclado.
<code>highlightthickness</code>	int	Espessura do retangulo de destaque de foco.
<code>takefocus</code>	bool	Define se o widget recebe foco ao navegar com a tecla Tab.

Exemplos de codigo

Exemplo 1: Grupo de opcoes


```
import tkinter as tk

root = tk.Tk()
genero = tk.StringVar(value='M')
for txt, val in [('Masculino', 'M'), ('Feminino', 'F'), ('Outro', 'O')]:
    tk.Radiobutton(root, text=txt, variable=genero, value=val).pack(anchor='w')
tk.Button(root, text='Ver', command=lambda: print(genero.get())).pack()
root.mainloop()
```

Todos os botoes usam a mesma variavel; apenas um pode estar ativo por vez.

Tela de exemplo: A tela mostra um grupo de opcoes exclusivas onde apenas uma fica marcada.

Listbox (Lista)

Importacao: import tkinter as tk

O Listbox exibe uma lista de itens de texto que podem ser selecionados. Suporta selecao unica ou multipla.

Itens sao adicionados com insert e lidos por indice. Combina bem com Scrollbar para listas longas.

Propriedades e metodos principais

Propriedade / Metodo	Tipo	Descricao
<code>selectmode</code>	constante	SINGLE, BROWSE, MULTIPLE ou EXTENDED.
<code>insert(indice, item)</code>	metodo	Adiciona item (use 'end' para o final).
<code>delete(ini, fim)</code>	metodo	Remove itens do intervalo.
<code>get(ini, fim)</code>	metodo	Retorna os itens do intervalo.
<code>curselection()</code>	metodo	Retorna os indices dos itens selecionados.
<code>activate(indice)</code>	metodo	Ativa o item indicado.

Opcoes comuns (compartilhadas com a maioria dos widgets)

Propriedade / Metodo	Tipo	Descricao
<code>background / bg</code>	cor	Cor de fundo do widget. Aceita nomes ('red', 'white') ou hexadecimal ('#2e2e2e').
<code>foreground / fg</code>	cor	Cor do texto (primeiro plano) exibido pelo widget.
<code>font</code>	tupla/str	Fonte do texto. Ex.: ('Arial', 12, 'bold') ou 'Helvetica 10 italic'.
<code>width</code>	int	Largura do widget. Em texto, medida em caracteres; em canvas/imagem, em pixels.
<code>height</code>	int	Altura do widget. Em texto, medida em linhas; em canvas/imagem, em pixels.
<code>relief</code>	constante	Estilo da borda: FLAT, RAISED, SUNKEN, GROOVE, RIDGE, SOLID.
<code>borderwidth / bd</code>	int	Espessura da borda em pixels.
<code>cursor</code>	str	Formato do cursor ao passar sobre o widget. Ex.: 'hand2', 'watch', 'cross'.
<code>padx</code>	int	Espacamento interno horizontal entre conteudo e a borda.
<code>pady</code>	int	Espacamento interno vertical entre conteudo e a borda.
<code>state</code>	constante	Estado do widget: NORMAL, DISABLED ou ACTIVE.
<code>highlightbackground</code>	cor	Cor do retangulo de foco quando o widget NAO tem foco.
<code>highlightcolor</code>	cor	Cor do retangulo de foco quando o widget tem o foco do teclado.
<code>highlightthickness</code>	int	Espessura do retangulo de destaque de foco.
<code>takefocus</code>	bool	Define se o widget recebe foco ao navegar com a tecla Tab.

Exemplos de codigo

Exemplo 1: Lista com adicao e remocao

```
import tkinter as tk

root = tk.Tk()
lista = tk.Listbox(root, height=6)
for fruta in ['Maca', 'Banana', 'Uva']:
    lista.insert('end', fruta)
lista.pack(padx=10, pady=10)

def remover():
    sel = lista.curselection()
    if sel:
        lista.delete(sel[0])

tk.Button(root, text='Remover selecionado', command=remover).pack()
root.mainloop()
```

curselection retorna os indices marcados, usados para remover o item escolhido.

Tela de exemplo: A tela mostra uma lista de frutas com um botao que remove o item selecionado.

Scale (Controle Deslizante)

Importacao: import tkinter as tk

O Scale cria um controle deslizante para escolher um valor numerico dentro de um intervalo, util para volume, brilho e configuracoes graduais.

Pode ser horizontal ou vertical e dispara um comando a cada movimento.

Propriedades e metodos principais

Propriedade / Metodo	Tipo	Descricao
from_	num	Valor inicial do intervalo.
to	num	Valor final do intervalo.
orient	constante	HORIZONTAL ou VERTICAL.
resolution	num	Incremento minimo entre valores.
tickinterval	num	Espacamento das marcas numericas.
command	funcao	Funcao que recebe o novo valor.
get()	metodo	Retorna o valor atual.
set(v)	metodo	Define o valor atual.

Opcoes comuns (compartilhadas com a maioria dos widgets)

Propriedade / Metodo	Tipo	Descricao
background / bg	cor	Cor de fundo do widget. Aceita nomes ('red', 'white') ou hexadecimal ('#2e2e2e').
foreground / fg	cor	Cor do texto (primeiro plano) exibido pelo widget.
font	tupla/str	Fonte do texto. Ex.: ('Arial', 12, 'bold') ou 'Helvetica 10 italic'.
width	int	Largura do widget. Em texto, medida em caracteres; em canvas/imagem, em pixels.
height	int	Altura do widget. Em texto, medida em linhas; em canvas/imagem, em pixels.
relief	constante	Estilo da borda: FLAT, RAISED, SUNKEN, GROOVE, RIDGE, SOLID.
borderwidth / bd	int	Espessura da borda em pixels.
cursor	str	Formato do cursor ao passar sobre o widget. Ex.: 'hand2', 'watch', 'cross'.
padx	int	Espacamento interno horizontal entre conteudo e a borda.
pady	int	Espacamento interno vertical entre conteudo e a borda.
state	constante	Estado do widget: NORMAL, DISABLED ou ACTIVE.
highlightbackground	cor	Cor do retangulo de foco quando o widget NAO tem foco.
highlightcolor	cor	Cor do retangulo de foco quando o widget tem o foco do teclado.
highlightthickness	int	Espessura do retangulo de destaque de foco.

Propriedade / Metodo	Tipo	Descricao
<code>takefocus</code>	bool	Define se o widget recebe foco ao navegar com a tecla Tab.

Exemplos de código

Exemplo 1: Slider de volume

```
import tkinter as tk

root = tk.Tk()
def mudou(v):
    print('Volume:', v)

tk.Scale(root, from_=0, to=100, orient='horizontal',
        label='Volume', command=mudou).pack(padx=20, pady=20)
root.mainloop()
```

O command recebe automaticamente o valor atual sempre que o controle se move.

Tela de exemplo: A tela mostra um controle deslizante horizontal de 0 a 100 com rotulo de volume.

Spinbox (Seletor Numerico)

Importacao: import tkinter as tk

O Spinbox e um campo numerico com setas para incrementar e decrementar valores dentro de um intervalo definido.

Aceita tambem uma lista de valores predefinidos atraves da opcao values.

Propriedades e metodos principais

Propriedade / Metodo	Tipo	Descricao
<code>from_ / to</code>	num	Intervalo de valores.
<code>increment</code>	num	Passo de incremento.
<code>values</code>	tupla	Lista de valores possiveis.
<code>textvariable</code>	Var	Variavel ligada ao valor.
<code>wrap</code>	bool	Se True, retorna ao inicio ao passar do limite.

Opcoes comuns (compartilhadas com a maioria dos widgets)

Propriedade / Metodo	Tipo	Descricao
<code>background / bg</code>	cor	Cor de fundo do widget. Aceita nomes ('red', 'white') ou hexadecimal ('#2e2e2e').
<code>foreground / fg</code>	cor	Cor do texto (primeiro plano) exibido pelo widget.
<code>font</code>	tupla/str	Fonte do texto. Ex.: ('Arial', 12, 'bold') ou 'Helvetica 10 italic'.
<code>width</code>	int	Largura do widget. Em texto, medida em caracteres; em canvas/imagem, em pixels.
<code>height</code>	int	Altura do widget. Em texto, medida em linhas; em canvas/imagem, em pixels.
<code>relief</code>	constante	Estilo da borda: FLAT, RAISED, SUNKEN, GROOVE, RIDGE, SOLID.
<code>borderwidth / bd</code>	int	Espessura da borda em pixels.
<code>cursor</code>	str	Formato do cursor ao passar sobre o widget. Ex.: 'hand2', 'watch', 'cross'.
<code>padx</code>	int	Espacamento interno horizontal entre conteudo e a borda.
<code>pady</code>	int	Espacamento interno vertical entre conteudo e a borda.
<code>state</code>	constante	Estado do widget: NORMAL, DISABLED ou ACTIVE.
<code>highlightbackground</code>	cor	Cor do retangulo de foco quando o widget NAO tem foco.
<code>highlightcolor</code>	cor	Cor do retangulo de foco quando o widget tem o foco do teclado.
<code>highlightthickness</code>	int	Espessura do retangulo de destaque de foco.
<code>takefocus</code>	bool	Define se o widget recebe foco ao navegar com a tecla Tab.

Exemplos de codigo

Exemplo 1: Selecionar quantidade

```
import tkinter as tk

root = tk.Tk()
qtd = tk.Spinbox(root, from_=1, to=10, width=5)
qtd.pack(padx=20, pady=20)
tk.Button(root, text='Ver', command=lambda: print(qtd.get())).pack()
root.mainloop()
```

As setas alteram o valor entre 1 e 10; get() retorna o numero escolhido.

Tela de exemplo: A tela mostra um seletor numerico de 1 a 10 com setas de incremento.

Menu (Menus)

Importacao: import tkinter as tk

O Menu cria barras de menu, menus suspensos e menus de contexto. E o padrao para organizar comandos em aplicacoes desktop.

Um menu principal e associado a janela; submenus sao adicionados como cascatas.

Propriedades e metodos principais

Propriedade / Metodo	Tipo	Descricao
<code>add_command(...)</code>	metodo	Adiciona um item clicavel ao menu.
<code>add_cascade(...)</code>	metodo	Adiciona um submenu.
<code>add_separator()</code>	metodo	Adiciona uma linha separadora.
<code>add_checkbutton(...)</code>	metodo	Item alternavel ligado a uma variavel.
<code>add_radiobutton(...)</code>	metodo	Item de opcao exclusiva.
<code>tearoff</code>	int	0 desativa o destacamento do menu.

Opcoes comuns (compartilhadas com a maioria dos widgets)

Propriedade / Metodo	Tipo	Descricao
<code>background / bg</code>	cor	Cor de fundo do widget. Aceita nomes ('red', 'white') ou hexadecimal ('#2e2e2e').
<code>foreground / fg</code>	cor	Cor do texto (primeiro plano) exibido pelo widget.
<code>font</code>	tupla/str	Fonte do texto. Ex.: ('Arial', 12, 'bold') ou 'Helvetica 10 italic'.
<code>width</code>	int	Largura do widget. Em texto, medida em caracteres; em canvas/imagem, em pixels.
<code>height</code>	int	Altura do widget. Em texto, medida em linhas; em canvas/imagem, em pixels.
<code>relief</code>	constante	Estilo da borda: FLAT, RAISED, SUNKEN, GROOVE, RIDGE, SOLID.
<code>borderwidth / bd</code>	int	Espessura da borda em pixels.
<code>cursor</code>	str	Formato do cursor ao passar sobre o widget. Ex.: 'hand2', 'watch', 'cross'.
<code>padx</code>	int	Espacamento interno horizontal entre conteudo e a borda.
<code>pady</code>	int	Espacamento interno vertical entre conteudo e a borda.
<code>state</code>	constante	Estado do widget: NORMAL, DISABLED ou ACTIVE.
<code>highlightbackground</code>	cor	Cor do retangulo de foco quando o widget NAO tem foco.
<code>highlightcolor</code>	cor	Cor do retangulo de foco quando o widget tem o foco do teclado.
<code>highlightthickness</code>	int	Espessura do retangulo de destaque de foco.
<code>takefocus</code>	bool	Define se o widget recebe foco ao navegar com a tecla Tab.

Exemplos de codigo

Exemplo 1: Barra de menu

```
import tkinter as tk

root = tk.Tk()
barra = tk.Menu(root)
arquivo = tk.Menu(barra, tearoff=0)
arquivo.add_command(label='Novo', command=lambda: print('Novo'))
arquivo.add_command(label='Abrir', command=lambda: print('Abrir'))
arquivo.add_separator()
arquivo.add_command(label='Sair', command=root.destroy)
barra.add_cascade(label='Arquivo', menu=arquivo)
root.config(menu=barra)
root.mainloop()
```

A barra é definida com `config(menu=...)` e recebe um submenu Arquivo com comandos.

Tela de exemplo: A tela mostra uma barra de menu com o menu Arquivo contendo Novo, Abrir e Sair.

Canvas (Tela de Desenho)

Importacao: import tkinter as tk

O Canvas e uma area de desenho 2D onde voce posiciona formas, linhas, textos e imagens por coordenadas. E a base para graficos, jogos e diagramas.

Cada item desenhado recebe um identificador que permite mover, alterar ou remover o objeto posteriormente.

Propriedades e metodos principais

Propriedade / Metodo	Tipo	Descricao
<code>create_line(...)</code>	metodo	Desenha uma linha entre pontos.
<code>create_rectangle(...)</code>	metodo	Desenha um retangulo.
<code>create_oval(...)</code>	metodo	Desenha elipse/circulo.
<code>create_text(...)</code>	metodo	Escreve texto em uma posicao.
<code>create_image(...)</code>	metodo	Coloca uma imagem.
<code>move(id, dx, dy)</code>	metodo	Movimenta um item pelo deslocamento informado.
<code>itemconfig(id, ...)</code>	metodo	Altera propriedades de um item.
<code>coords(id, ...)</code>	metodo	Le ou redefine as coordenadas de um item.

Opcoes comuns (compartilhadas com a maioria dos widgets)

Propriedade / Metodo	Tipo	Descricao
<code>background / bg</code>	cor	Cor de fundo do widget. Aceita nomes ('red', 'white') ou hexadecimal ('#2e2e2e').
<code>foreground / fg</code>	cor	Cor do texto (primeiro plano) exibido pelo widget.
<code>font</code>	tupla/str	Fonte do texto. Ex.: ('Arial', 12, 'bold') ou 'Helvetica 10 italic'.
<code>width</code>	int	Largura do widget. Em texto, medida em caracteres; em canvas/imagem, em pixels.
<code>height</code>	int	Altura do widget. Em texto, medida em linhas; em canvas/imagem, em pixels.
<code>relief</code>	constante	Estilo da borda: FLAT, RAISED, SUNKEN, GROOVE, RIDGE, SOLID.
<code>borderwidth / bd</code>	int	Espessura da borda em pixels.
<code>cursor</code>	str	Formato do cursor ao passar sobre o widget. Ex.: 'hand2', 'watch', 'cross'.
<code>padx</code>	int	Espacamento interno horizontal entre conteudo e a borda.
<code>pady</code>	int	Espacamento interno vertical entre conteudo e a borda.
<code>state</code>	constante	Estado do widget: NORMAL, DISABLED ou ACTIVE.
<code>highlightbackground</code>	cor	Cor do retangulo de foco quando o widget NAO tem foco.
<code>highlightcolor</code>	cor	Cor do retangulo de foco quando o widget tem o foco do teclado.
<code>highlightthickness</code>	int	Espessura do retangulo de destaque de foco.

Propriedade / Metodo	Tipo	Descricao
<code>takefocus</code>	bool	Define se o widget recebe foco ao navegar com a tecla Tab.

Exemplos de código

Exemplo 1: Desenhar formas

```
import tkinter as tk

root = tk.Tk()
c = tk.Canvas(root, width=300, height=200, bg='white')
c.pack()
c.create_rectangle(20, 20, 120, 100, fill='#3b82f6')
c.create_oval(150, 20, 250, 120, fill='#ef4444')
c.create_line(20, 150, 280, 150, width=3)
c.create_text(150, 180, text='Canvas', font=('Arial', 14))
root.mainloop()
```

Cada metodo `create_` desenha uma forma usando coordenadas (x1,y1,x2,y2).

Exemplo 2: Animacao simples

```
import tkinter as tk

root = tk.Tk()
c = tk.Canvas(root, width=300, height=120, bg='white')
c.pack()
bola = c.create_oval(10, 40, 50, 80, fill='green')

def mover():
    c.move(bola, 5, 0)
    if c.coords(bola)[2] < 300:
        root.after(50, mover)

mover()
root.mainloop()
```

after agenda a proxima posicao a cada 50ms, criando uma animacao da bola.

Tela de exemplo: A tela mostra formas coloridas desenhadas e uma bola que se move da esquerda para a direita.

Parte III – Widgets Tematicos (ttk)

O modulo ttk traz widgets com aparencia moderna e suporte a temas. Sempre que possivel, prefira os widgets ttk aos equivalentes classicos para obter uma interface mais elegante e consistente com o sistema operacional.

ttk.Style (Estilos e Temas)

Importacao: from tkinter import ttk

O modulo ttk (themed Tkinter) traz widgets com aparencia moderna que respeitam o tema do sistema operacional. A classe Style centraliza a customizacao desses widgets.

Com Style definimos temas, cores, fontes e estados (hover, pressionado) de forma consistente para todos os widgets ttk.

Propriedades e metodos principais

Propriedade / Metodo	Tipo	Descricao
<code>theme_use(nome)</code>	metodo	Aplica um tema: 'clam', 'alt', 'default', 'classic'.
<code>theme_names()</code>	metodo	Lista os temas disponiveis.
<code>configure(estilo, ...)</code>	metodo	Define propriedades de um estilo nomeado.
<code>map(estilo, ...)</code>	metodo	Define propriedades por estado (active, disabled).
<code>layout(estilo)</code>	metodo	Inspeciona/redefine a estrutura visual de um estilo.

Exemplos de codigo

Exemplo 1: Personalizar botoes ttk

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()
estilo = ttk.Style()
estilo.theme_use('clam')
estilo.configure('TButton', font=('Arial', 12), padding=10)
estilo.map('TButton', background=[('active', '#3b82f6')])
ttk.Button(root, text='Botao Estilizado').pack(padx=20, pady=20)
root.mainloop()
```

configure define a aparencia base e map altera a cor quando o botao esta ativo.

Tela de exemplo: A tela mostra um botao ttk com tema 'clam' que muda de cor ao passar o mouse.

ttk.Combobox (Caixa de Combinacao)

Importacao: from tkinter import ttk

O Combobox combina um campo de texto com uma lista suspensa, permitindo escolher um item ou digitar um valor.

Os valores sao definidos na opcao values e o item escolhido e lido com get().

Propriedades e metodos principais

Propriedade / Metodo	Tipo	Descricao
values	lista	Itens exibidos na lista suspensa.
textvariable	Var	Variavel ligada ao valor selecionado.
state	constante	'readonly' impede digitacao manual.
current(indice)	metodo	Seleciona um item pela posicao.
get()	metodo	Retorna o valor selecionado.
<<ComboboxSelected>>	evento	Disparado quando um item e escolhido.

Exemplos de codigo

Exemplo 1: Selecao de cidade

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()
cb = ttk.Combobox(root, values=['Sao Paulo', 'Rio', 'Belo Horizonte'],
                  state='readonly')
cb.current(0)
cb.pack(padx=20, pady=20)
cb.bind('<<ComboboxSelected>>', lambda e: print(cb.get()))
root.mainloop()
```

state='readonly' obriga a escolha pela lista; o evento imprime a selecao.

Tela de exemplo: A tela mostra uma caixa de combinacao com cidades selecionaveis.

ttk.Treeview (Tabela/Arvore)

Importacao: from tkinter import ttk

O Treeview exibe dados em formato de tabela com colunas ou em estrutura hierarquica de arvore. E o widget ideal para listar registros, planilhas e arquivos.

Colunas sao definidas com columns/heading e linhas inseridas com insert.

Propriedades e metodos principais

Propriedade / Metodo	Tipo	Descricao
<code>columns</code>	tupla	Identificadores das colunas.
<code>heading(col, text=)</code>	metodo	Define o titulo de uma coluna.
<code>column(col, width=)</code>	metodo	Configura largura/alinhamento da coluna.
<code>insert(pai, indice, ...)</code>	metodo	Insera uma linha (use " como raiz).
<code>selection()</code>	metodo	Retorna os itens selecionados.
<code>delete(item)</code>	metodo	Remove um item.
<code>show</code>	constante	'headings' oculta a coluna de arvore.

Exemplos de codigo

Exemplo 1: Tabela de produtos

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()
tv = ttk.Treeview(root, columns=('nome', 'preco'), show='headings')
tv.heading('nome', text='Produto')
tv.heading('preco', text='Preco')
tv.insert('', 'end', values=('Teclado', 'R$ 120'))
tv.insert('', 'end', values=('Mouse', 'R$ 80'))
tv.pack(padx=10, pady=10, fill='both', expand=True)
root.mainloop()
```

show='headings' cria uma tabela limpa; cada insert adiciona uma linha de dados.

Tela de exemplo: A tela mostra uma tabela com colunas Produto e Preco preenchida com itens.

ttk.Notebook (Abas)

Importacao: from tkinter import ttk

O Notebook cria uma interface com abas (guias), permitindo alternar entre paginas de conteudo no mesmo espaco.

Cada aba recebe um frame de conteudo adicionado com add.

Propriedades e metodos principais

Propriedade / Metodo	Tipo	Descricao
<code>add(frame, text=)</code>	metodo	Adiciona uma aba com o frame e o titulo.
<code>select(aba)</code>	metodo	Seleciona uma aba programaticamente.
<code>index('current')</code>	metodo	Retorna o indice da aba ativa.
<code><<NotebookTabChanged>></code>	evento	Disparado ao trocar de aba.

Exemplos de codigo

Exemplo 1: Interface com abas

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()
nb = ttk.Notebook(root)
aba1 = ttk.Frame(nb); aba2 = ttk.Frame(nb)
tk.Label(aba1, text='Conteudo da Aba 1').pack(pady=20)
tk.Label(aba2, text='Conteudo da Aba 2').pack(pady=20)
nb.add(aba1, text='Inicio')
nb.add(aba2, text='Config')
nb.pack(fill='both', expand=True)
root.mainloop()
```

Cada frame vira uma aba navegavel adicionada com add.

Tela de exemplo: A tela mostra duas abas (Inicio e Config) com conteudos diferentes.

ttk.Progressbar (Barra de Progresso)

Importacao: from tkinter import ttk

A Progressbar indica o andamento de uma tarefa, no modo determinado (porcentagem conhecida) ou indeterminado (animacao continua).

O valor e controlado por value/maximum ou pelos metodos start/stop.

Propriedades e metodos principais

Propriedade / Metodo	Tipo	Descricao
<code>mode</code>	constante	'determinate' ou 'indeterminate'.
<code>value</code>	num	Progresso atual.
<code>maximum</code>	num	Valor maximo (100 por padrao).
<code>start(ms)</code>	metodo	Inicia a animacao indeterminada.
<code>stop()</code>	metodo	Para a animacao.
<code>step(qtd)</code>	metodo	Incrementa o valor.

Exemplos de codigo

Exemplo 1: Progresso determinado

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()
pb = ttk.Progressbar(root, length=250, mode='determinate', maximum=100)
pb.pack(padx=20, pady=20)

def avancar():
    if pb['value'] < 100:
        pb['value'] += 10
        root.after(300, avancar)

avancar()
root.mainloop()
```

O valor aumenta 10 a cada 300ms ate completar 100%.

Tela de exemplo: A tela mostra uma barra de progresso que avanca ate 100%.

Parte IV – Eventos e Vinculacoes (bind)

Eventos são ações do usuário ou do sistema, como cliques do mouse, teclas pressionadas, movimento do ponteiro e redimensionamento de janelas. O método `bind` associa uma função (manipulador) a um evento específico de um widget.

A sintaxe é `widget.bind('<Evento>', funcao)`. A função recebe um objeto `event` com informações úteis, como `event.x`, `event.y` (posição do mouse), `event.char` e `event.keysym` (tecla pressionada) e `event.widget` (widget que gerou o evento).

Exemplo: capturando clique e teclado

```
import tkinter as tk
root = tk.Tk(); root.geometry('300x200')
label = tk.Label(root, text='Mova o mouse / clique')
label.pack(expand=True, fill='both')

def clicou(event):
    label.config(text=f'Clique em ({event.x}, {event.y})')

def tecla(event):
    print('Tecla:', event.keysym)

label.bind('<Button-1>', clicou)
root.bind('<Key>', tecla)
root.mainloop()
```

Referencia de eventos comuns

Sequencias de eventos mais usadas

Propriedade / Metodo	Tipo	Descricao
<code><Button-1></code>	evento	Clique com o botao esquerdo do mouse.
<code><Button-2></code>	evento	Clique com o botao do meio do mouse.
<code><Button-3></code>	evento	Clique com o botao direito do mouse.
<code><Double-Button-1></code>	evento	Clique duplo com o botao esquerdo.
<code><ButtonRelease-1></code>	evento	Soltar o botao esquerdo do mouse.
<code><B1-Motion></code>	evento	Mover o mouse com o botao esquerdo pressionado (arrastar).
<code><Motion></code>	evento	Movimento do mouse sobre o widget.
<code><Enter></code>	evento	O ponteiro do mouse entra na area do widget.
<code><Leave></code>	evento	O ponteiro do mouse sai da area do widget.
<code><MouseWheel></code>	evento	Rolagem da roda do mouse (Windows/Mac).
<code><Key></code>	evento	Qualquer tecla pressionada.
<code><Return></code>	evento	Tecla Enter pressionada.
<code><Escape></code>	evento	Tecla Esc pressionada.

Propriedade / Metodo	Tipo	Descricao
<space>	evento	Barra de espaco pressionada.
<BackSpace>	evento	Tecla Backspace pressionada.
<Tab>	evento	Tecla Tab pressionada.
<Up> <Down> <Left> <Right>	evento	Teclas de seta direcionais.
<Control-c>	evento	Combinacao Ctrl + C.
<Control-s>	evento	Combinacao Ctrl + S (salvar).
<Shift-Up>	evento	Shift combinado com seta para cima.
<Alt-F4>	evento	Combinacao Alt + F4.
<FocusIn>	evento	O widget recebe o foco do teclado.
<FocusOut>	evento	O widget perde o foco do teclado.
<Configure>	evento	O widget e redimensionado ou movido.
<Destroy>	evento	O widget e destruido.
<<ComboboxSelected>>	evento	Um item do Combobox foi selecionado.
<<NotebookTabChanged>>	evento	A aba ativa do Notebook mudou.
<<ListboxSelect>>	evento	A selecao da Listbox mudou.

Dica: Eventos com << >> duplo, como <<ComboboxSelected>>, sao eventos virtuais gerados por widgets especificos.

Parte V – Caixas de Dialogo e Mensagens

O Tkinter oferece modulos prontos para exibir mensagens, pedir confirmacoes e abrir janelas de selecao de arquivos e cores. Eles economizam muito codigo e seguem a aparencia nativa do sistema.

messagebox – mensagens

Funcoes do messagebox

Propriedade / Metodo	Tipo	Descricao
<code>showinfo(titulo, msg)</code>	funcao	Exibe uma mensagem informativa.
<code>showwarning(titulo, msg)</code>	funcao	Exibe um aviso.
<code>showerror(titulo, msg)</code>	funcao	Exibe um erro.
<code>askquestion(titulo, msg)</code>	funcao	Pergunta sim/nao (retorna 'yes'/'no').
<code>askyesno(titulo, msg)</code>	funcao	Retorna True/False.
<code>askokcancel(titulo, msg)</code>	funcao	Retorna True/False (OK/Cancelar).
<code>askretrycancel(titulo, msg)</code>	funcao	Tentar novamente/Cancelar.

Exemplo: confirmacao com messagebox

```
import tkinter as tk
from tkinter import messagebox
root = tk.Tk()
def confirmar():
    if messagebox.askyesno('Confirmar', 'Deseja continuar?'):
        messagebox.showinfo('OK', 'Voce confirmou!')
    else:
        messagebox.showwarning('Cancelado', 'Operacao cancelada.')
tk.Button(root, text='Testar', command=confirmar).pack(padx=40, pady=40)
root.mainloop()
```

filedialog – arquivos

Funcoes do filedialog

Propriedade / Metodo	Tipo	Descricao
<code>askopenfilename()</code>	funcao	Seleciona um arquivo para abrir (retorna o caminho).
<code>asksaveasfilename()</code>	funcao	Define caminho/nome para salvar.
<code>askopenfilenames()</code>	funcao	Seleciona varios arquivos.
<code>askdirectory()</code>	funcao	Seleciona uma pasta.

Propriedade / Metodo	Tipo	Descricao
<code>filetypes</code>	lista	Filtra extensoes: [('Texto', '*.txt')].

colorchooser – cores

Exemplo: seletor de cores

```
import tkinter as tk
from tkinter import colorchooser
root = tk.Tk()
def escolher():
    cor = colorchooser.askcolor(title='Escolha uma cor')
    if cor[1]:
        root.config(bg=cor[1])
tk.Button(root, text='Cor de fundo', command=escolher).pack(padx=40, pady=40)
root.mainloop()
```

Parte VI – CustomTkinter

O CustomTkinter moderniza completamente a aparencia das interfaces Tkinter. Ele oferece widgets com cantos arredondados, suporte nativo a modo claro/escuro, temas de cores e escalonamento automatico para telas de alta resolucao.

A maioria dos conceitos do Tkinter continua valida: gerenciadores de geometria, variaveis de controle, callbacks e eventos funcionam da mesma forma. A principal diferenca esta nos nomes dos widgets (prefixados com CTK) e em opcoes visuais adicionais como `corner_radius`, `fg_color` e `hover_color`.

Instalacao: `pip install customtkinter`

CTk e Configuracao Inicial

Importacao: import customtkinter as ctk

O CustomTkinter e uma biblioteca construida sobre o Tkinter que oferece widgets modernos, cantos arredondados, modo claro/escuro e temas de cores. A janela principal e criada com `ctk.CTk()` no lugar de `tk.Tk()`.

Antes de criar widgets, definimos o modo de aparencia e o tema de cores globalmente com `set_appearance_mode` e `set_default_color_theme`.

Propriedades e metodos principais

Propriedade / Metodo	Tipo	Descricao
<code>set_appearance_mode(m)</code>	funcao	'light', 'dark' ou 'system'.
<code>set_default_color_theme(t)</code>	funcao	'blue', 'green' ou 'dark-blue'.
<code>set_widget_scaling(f)</code>	funcao	Escala todos os widgets por um fator.
<code>title(texto)</code>	metodo	Define o titulo da janela.
<code>geometry('LxA')</code>	metodo	Define o tamanho da janela.
<code>mainloop()</code>	metodo	Inicia o laco de eventos.

Opcoes comuns (compartilhadas com a maioria dos widgets)

Propriedade / Metodo	Tipo	Descricao
<code>master</code>	widget	Widget pai onde o componente sera colocado.
<code>width / height</code>	int	Largura e altura em pixels.
<code>corner_radius</code>	int	Raio dos cantos arredondados (visual moderno).
<code>fg_color</code>	cor/tupla	Cor de preenchimento. Aceita ('claro','escuro') por modo de tema.
<code>bg_color</code>	cor	Cor de fundo atras do widget.
<code>border_width</code>	int	Espessura da borda.
<code>border_color</code>	cor	Cor da borda.
<code>text_color</code>	cor	Cor do texto.
<code>font</code>	CTkFont/tupla	Fonte usada no texto.
<code>hover_color</code>	cor	Cor exibida ao passar o mouse (botoes/itens).
<code>state</code>	str	'normal' ou 'disabled'.

Exemplos de codigo

Exemplo 1: Janela CustomTkinter

```
import customtkinter as ctk

ctk.set_appearance_mode('dark')
ctk.set_default_color_theme('blue')

app = ctk.CTk()
app.title('App Moderno')
app.geometry('500x400')

ctk.CTkLabel(app, text='Ola, CustomTkinter!',
              font=ctk.CTkFont(size=20, weight='bold')).pack(pady=40)
app.mainloop()
```

Configura modo escuro, tema azul e cria uma janela moderna com um rotulo.

Exemplo 2: Alternar modo claro/escuro

```
import customtkinter as ctk

app = ctk.CTk(); app.geometry('300x200')
def alternar():
    atual = ctk.get_appearance_mode()
    ctk.set_appearance_mode('light' if atual == 'Dark' else 'dark')

ctk.CTkButton(app, text='Alternar tema', command=alternar).pack(pady=60)
app.mainloop()
```

get_appearance_mode retorna o modo atual, usado para inverter o tema.

Tela de exemplo: A tela mostra uma janela escura e moderna com um rotulo central e um botao que alterna entre tema claro e escuro.

CTkButton (Botao Moderno)

Importacao: import customtkinter as ctk

O CTkButton e a versao moderna do botao, com cantos arredondados, cores de hover e suporte a imagens. Aceita os mesmos conceitos de command do Tkinter.

Permite ampla customizacao visual: cores por estado, raio dos cantos e bordas.

Propriedades e metodos principais

Propriedade / Metodo	Tipo	Descricao
<code>text</code>	str	Texto do botao.
<code>command</code>	funcao	Funcao executada no clique.
<code>fg_color</code>	cor	Cor de fundo do botao.
<code>hover_color</code>	cor	Cor ao passar o mouse.
<code>corner_radius</code>	int	Arredondamento dos cantos.
<code>image</code>	CTkImage	Imagem exibida no botao.
<code>text_color</code>	cor	Cor do texto.

Opcoes comuns (compartilhadas com a maioria dos widgets)

Propriedade / Metodo	Tipo	Descricao
<code>master</code>	widget	Widget pai onde o componente sera colocado.
<code>width / height</code>	int	Largura e altura em pixels.
<code>corner_radius</code>	int	Raio dos cantos arredondados (visual moderno).
<code>fg_color</code>	cor/tupla	Cor de preenchimento. Aceita ('claro','escuro') por modo de tema.
<code>bg_color</code>	cor	Cor de fundo atras do widget.
<code>border_width</code>	int	Espessura da borda.
<code>border_color</code>	cor	Cor da borda.
<code>text_color</code>	cor	Cor do texto.
<code>font</code>	CTkFont/tupla	Fonte usada no texto.
<code>hover_color</code>	cor	Cor exibida ao passar o mouse (botoes/itens).
<code>state</code>	str	'normal' ou 'disabled'.

Exemplos de codigo

Exemplo 1: Botao customizado

```
import customtkinter as ctk

app = ctk.CTk(); app.geometry('300x200')
ctk.CTkButton(app, text='Salvar', corner_radius=20,
              fg_color='#22c55e', hover_color='#16a34a',
              command=lambda: print('Salvo!')).pack(pady=60)
app.mainloop()
```

Define cores personalizadas, cantos arredondados e um callback.

Tela de exemplo: A tela mostra um botao verde arredondado com efeito de hover.

CTkLabel (Rotulo Moderno)

Importacao: import customtkinter as ctk

O CTKLabel exibe texto e imagens com a estetica do CustomTkinter, incluindo cantos arredondados opcionais e cores por modo de tema.

Propriedades e metodos principais

Propriedade / Metodo	Tipo	Descricao
<code>text</code>	str	Texto exibido.
<code>font</code>	CTkFont	Fonte do texto.
<code>text_color</code>	cor	Cor do texto.
<code>fg_color</code>	cor	Cor de fundo (use 'transparent' para sem fundo).
<code>corner_radius</code>	int	Arredondamento do fundo.
<code>image</code>	CTkImage	Imagem exibida.

Opcoes comuns (compartilhadas com a maioria dos widgets)

Propriedade / Metodo	Tipo	Descricao
<code>master</code>	widget	Widget pai onde o componente sera colocado.
<code>width / height</code>	int	Largura e altura em pixels.
<code>corner_radius</code>	int	Raio dos cantos arredondados (visual moderno).
<code>fg_color</code>	cor/tupla	Cor de preenchimento. Aceita ('claro','escuro') por modo de tema.
<code>bg_color</code>	cor	Cor de fundo atras do widget.
<code>border_width</code>	int	Espessura da borda.
<code>border_color</code>	cor	Cor da borda.
<code>text_color</code>	cor	Cor do texto.
<code>font</code>	CTkFont/tupla	Fonte usada no texto.
<code>hover_color</code>	cor	Cor exibida ao passar o mouse (botoes/itens).
<code>state</code>	str	'normal' ou 'disabled'.

Exemplos de codigo

Exemplo 1: Rotulo com fundo

```
import customtkinter as ctk

app = ctk.CTk(); app.geometry('320x160')
ctk.CTkLabel(app, text='Destaque', fg_color='#3b82f6',
             corner_radius=10, text_color='white',
             font=ctk.CTkFont(size=18)).pack(pady=50, padx=20)
app.mainloop()
```

Um rotulo com fundo colorido e cantos arredondados funciona como uma etiqueta.

Tela de exemplo: A tela mostra um rotulo com fundo azul arredondado e texto branco.

CTkEntry (Campo Moderno)

Importacao: import customtkinter as ctk

O CTkEntry e o campo de entrada de texto moderno, com placeholder nativo, cantos arredondados e suporte a mascara de senha.

Propriedades e metodos principais

Propriedade / Metodo	Tipo	Descricao
<code>placeholder_text</code>	str	Texto de dica exibido quando vazio.
<code>show</code>	str	Caractere de mascara (senha).
<code>textvariable</code>	Var	Variavel ligada ao conteudo.
<code>corner_radius</code>	int	Arredondamento dos cantos.
<code>get()</code> / <code>insert()</code> / <code>delete()</code>	metodo	Manipulam o conteudo como no Entry.

Opcoes comuns (compartilhadas com a maioria dos widgets)

Propriedade / Metodo	Tipo	Descricao
<code>master</code>	widget	Widget pai onde o componente sera colocado.
<code>width</code> / <code>height</code>	int	Largura e altura em pixels.
<code>corner_radius</code>	int	Raio dos cantos arredondados (visual moderno).
<code>fg_color</code>	cor/tupla	Cor de preenchimento. Aceita ('claro','escuro') por modo de tema.
<code>bg_color</code>	cor	Cor de fundo atras do widget.
<code>border_width</code>	int	Espessura da borda.
<code>border_color</code>	cor	Cor da borda.
<code>text_color</code>	cor	Cor do texto.
<code>font</code>	CTkFont/tupla	Fonte usada no texto.
<code>hover_color</code>	cor	Cor exibida ao passar o mouse (botoes/itens).
<code>state</code>	str	'normal' ou 'disabled'.

Exemplos de codigo

Exemplo 1: Campo com placeholder

```
import customtkinter as ctk

app = ctk.CTk(); app.geometry('320x220')
email = ctk.CTkEntry(app, placeholder_text='Digite seu e-mail', width=240)
email.pack(pady=20)
senha = ctk.CTkEntry(app, placeholder_text='Senha', show='*', width=240)
senha.pack(pady=10)
ctk.CTkButton(app, text='Entrar',
               command=lambda: print(email.get())).pack(pady=20)
app.mainloop()
```

placeholder_text mostra dicas; show='' protege a senha.*

Tela de exemplo: A tela mostra um mini formulario de login com campos de e-mail e senha modernos.

CTkCheckBox (Caixa Moderna)

Importacao: import customtkinter as ctk

O CtkCheckBox e a caixa de selecao moderna, com animacao e cores customizaveis, ligada a uma variavel de controle.

Propriedades e metodos principais

Propriedade / Metodo	Tipo	Descricao
<code>text</code>	str	Texto da opcao.
<code>variable</code>	Var	Variavel que guarda o estado.
<code>onvalue</code> / <code>offvalue</code>	valor	Valores para marcado/desmarcado.
<code>command</code>	funcao	Funcao chamada ao alternar.
<code>checkbox_width</code> / <code>height</code>	int	Tamanho da caixa.

Opcoes comuns (compartilhadas com a maioria dos widgets)

Propriedade / Metodo	Tipo	Descricao
<code>master</code>	widget	Widget pai onde o componente sera colocado.
<code>width</code> / <code>height</code>	int	Largura e altura em pixels.
<code>corner_radius</code>	int	Raio dos cantos arredondados (visual moderno).
<code>fg_color</code>	cor/tupla	Cor de preenchimento. Aceita ('claro','escuro') por modo de tema.
<code>bg_color</code>	cor	Cor de fundo atras do widget.
<code>border_width</code>	int	Espessura da borda.
<code>border_color</code>	cor	Cor da borda.
<code>text_color</code>	cor	Cor do texto.
<code>font</code>	CTkFont/tupla	Fonte usada no texto.
<code>hover_color</code>	cor	Cor exibida ao passar o mouse (botoes/itens).
<code>state</code>	str	'normal' ou 'disabled'.

Exemplos de codigo

Exemplo 1: Caixa de selecao

```
import customtkinter as ctk

app = ctk.CTk(); app.geometry('300x150')
v = ctk.StringVar(value='off')
ctk.CTkCheckBox(app, text='Receber novidades', variable=v,
                onvalue='on', offvalue='off').pack(pady=40)
app.mainloop()
```

A variavel alterna entre 'on' e 'off' conforme o estado da caixa.

Tela de exemplo: A tela mostra uma caixa de selecao moderna com texto descritivo.

CTkRadioButton (Opcao Moderna)

Importacao: import customtkinter as ctk

O CTKRadioButton oferece opcoes exclusivas com aparencia moderna; botoes do mesmo grupo compartilham uma variavel.

Propriedades e metodos principais

Propriedade / Metodo	Tipo	Descricao
<code>text</code>	str	Texto da opcao.
<code>variable</code>	Var	Variavel compartilhada.
<code>value</code>	valor	Valor desta opcao.
<code>command</code>	funcao	Funcao ao selecionar.

Opcoes comuns (compartilhadas com a maioria dos widgets)

Propriedade / Metodo	Tipo	Descricao
<code>master</code>	widget	Widget pai onde o componente sera colocado.
<code>width / height</code>	int	Largura e altura em pixels.
<code>corner_radius</code>	int	Raio dos cantos arredondados (visual moderno).
<code>fg_color</code>	cor/tupla	Cor de preenchimento. Aceita ('claro','escuro') por modo de tema.
<code>bg_color</code>	cor	Cor de fundo atras do widget.
<code>border_width</code>	int	Espessura da borda.
<code>border_color</code>	cor	Cor da borda.
<code>text_color</code>	cor	Cor do texto.
<code>font</code>	CTkFont/tupla	Fonte usada no texto.
<code>hover_color</code>	cor	Cor exibida ao passar o mouse (botoes/itens).
<code>state</code>	str	'normal' ou 'disabled'.

Exemplos de codigo

Exemplo 1: Grupo de opcoes

```
import customtkinter as ctk

app = ctk.CTk(); app.geometry('260x200')
plano = ctk.StringVar(value='basico')
for txt in ['basico', 'pro', 'premium']:
    ctk.CTkRadioButton(app, text=txt.title(), variable=plano,
                       value=txt).pack(anchor='w', padx=30, pady=6)
app.mainloop()
```

Apenas um plano pode estar selecionado por vez no grupo.

Tela de exemplo: A tela mostra tres opcoes de plano onde apenas uma fica marcada.

CTkSwitch (Interruptor)

Importacao: import customtkinter as ctk

O CTkSwitch e um interruptor liga/desliga (toggle), comum em telas de configuracao para ativar/desativar recursos.

Propriedades e metodos principais

Propriedade / Metodo	Tipo	Descricao
<code>text</code>	str	Texto ao lado do interruptor.
<code>variable</code>	Var	Variavel de estado.
<code>onvalue</code> / <code>offvalue</code>	valor	Valores para ligado/desligado.
<code>command</code>	funcao	Funcao ao alternar.
<code>progress_color</code>	cor	Cor do trilho quando ligado.

Opcoes comuns (compartilhadas com a maioria dos widgets)

Propriedade / Metodo	Tipo	Descricao
<code>master</code>	widget	Widget pai onde o componente sera colocado.
<code>width</code> / <code>height</code>	int	Largura e altura em pixels.
<code>corner_radius</code>	int	Raio dos cantos arredondados (visual moderno).
<code>fg_color</code>	cor/tupla	Cor de preenchimento. Aceita ('claro','escuro') por modo de tema.
<code>bg_color</code>	cor	Cor de fundo atras do widget.
<code>border_width</code>	int	Espessura da borda.
<code>border_color</code>	cor	Cor da borda.
<code>text_color</code>	cor	Cor do texto.
<code>font</code>	CTkFont/tupla	Fonte usada no texto.
<code>hover_color</code>	cor	Cor exibida ao passar o mouse (botoes/itens).
<code>state</code>	str	'normal' ou 'disabled'.

Exemplos de codigo

Exemplo 1: Interruptor de notificacoes

```
import customtkinter as ctk

app = ctk.CTk(); app.geometry('300x150')
sw = ctk.CTkSwitch(app, text='Notificacoes',
                   command=lambda: print('Estado:', sw.get()))
sw.pack(pady=50)
app.mainloop()
```

get() retorna 1 quando ligado e 0 quando desligado.

Tela de exemplo: A tela mostra um interruptor estilizado para ativar notificacoes.

CTkSlider (Controle Deslizante)

Importacao: import customtkinter as ctk

O CTkSlider é o controle deslizante moderno para selecionar valores numéricos em um intervalo, com aparência suave e cores personalizáveis.

Propriedades e metodos principais

Propriedade / Metodo	Tipo	Descricao
<code>from_ / to</code>	num	Limites do intervalo.
<code>number_of_steps</code>	int	Quantidade de passos discretos.
<code>command</code>	funcao	Funcao que recebe o valor.
<code>progress_color</code>	cor	Cor da parte preenchida.
<code>get() / set(v)</code>	metodo	Le ou define o valor.

Opcoes comuns (compartilhadas com a maioria dos widgets)

Propriedade / Metodo	Tipo	Descricao
<code>master</code>	widget	Widget pai onde o componente sera colocado.
<code>width / height</code>	int	Largura e altura em pixels.
<code>corner_radius</code>	int	Raio dos cantos arredondados (visual moderno).
<code>fg_color</code>	cor/tupla	Cor de preenchimento. Aceita ('claro','escuro') por modo de tema.
<code>bg_color</code>	cor	Cor de fundo atras do widget.
<code>border_width</code>	int	Espessura da borda.
<code>border_color</code>	cor	Cor da borda.
<code>text_color</code>	cor	Cor do texto.
<code>font</code>	CTkFont/tupla	Fonte usada no texto.
<code>hover_color</code>	cor	Cor exibida ao passar o mouse (botoes/itens).
<code>state</code>	str	'normal' ou 'disabled'.

Exemplos de codigo

Exemplo 1: Slider com rotulo dinamico

```
import customtkinter as ctk

app = ctk.CTk(); app.geometry('320x180')
lbl = ctk.CTkLabel(app, text='50')
lbl.pack(pady=10)
ctk.CTkSlider(app, from_=0, to=100,
               command=lambda v: lbl.configure(text=str(int(v)))).pack(pady=20)
app.mainloop()
```

O command atualiza o rotulo com o valor atual sempre que o slider se move.

Tela de exemplo: A tela mostra um controle deslizante que atualiza um numero acima dele.

CTkProgressBar (Barra de Progresso)

Importacao: import customtkinter as ctk

A CTkProgressBar mostra progresso com valores de 0 a 1, em modo determinado ou indeterminado, com visual moderno.

Propriedades e metodos principais

Propriedade / Metodo	Tipo	Descricao
<code>mode</code>	str	'determinate' ou 'indeterminate'.
<code>set(v)</code>	metodo	Define o progresso (0.0 a 1.0).
<code>start() / stop()</code>	metodo	Controlam a animacao indeterminada.
<code>progress_color</code>	cor	Cor da barra preenchida.

Opcoes comuns (compartilhadas com a maioria dos widgets)

Propriedade / Metodo	Tipo	Descricao
<code>master</code>	widget	Widget pai onde o componente sera colocado.
<code>width / height</code>	int	Largura e altura em pixels.
<code>corner_radius</code>	int	Raio dos cantos arredondados (visual moderno).
<code>fg_color</code>	cor/tupla	Cor de preenchimento. Aceita ('claro','escuro') por modo de tema.
<code>bg_color</code>	cor	Cor de fundo atras do widget.
<code>border_width</code>	int	Espessura da borda.
<code>border_color</code>	cor	Cor da borda.
<code>text_color</code>	cor	Cor do texto.
<code>font</code>	CTkFont/tupla	Fonte usada no texto.
<code>hover_color</code>	cor	Cor exibida ao passar o mouse (botoes/itens).
<code>state</code>	str	'normal' ou 'disabled'.

Exemplos de codigo

Exemplo 1: Progresso animado

```
import customtkinter as ctk

app = ctk.CTk(); app.geometry('320x140')
pb = ctk.CTkProgressBar(app, width=240)
pb.set(0)
pb.pack(pady=40)
valor = {'v': 0.0}
def avancar():
    valor['v'] += 0.1
    pb.set(valor['v'])
    if valor['v'] < 1.0:
        app.after(300, avancar)
avancar()
app.mainloop()
```

set recebe valores entre 0 e 1; o progresso avança a cada 300ms.

Tela de exemplo: A tela mostra uma barra de progresso moderna que enche gradualmente.

CTkComboBox e CTkOptionMenu

Importacao: import customtkinter as ctk

O CTkComboBox combina campo editavel com lista suspensa; o CTkOptionMenu apresenta apenas um menu de opcoes para escolha. Ambos sao versoes modernas de seletores.

Propriedades e metodos principais

Propriedade / Metodo	Tipo	Descricao
<code>values</code>	lista	Itens disponiveis.
<code>command</code>	funcao	Funcao que recebe o valor escolhido.
<code>variable</code>	Var	Variavel ligada ao valor.
<code>get()</code> / <code>set(v)</code>	metodo	Le ou define a selecao.
<code>dropdown_fg_color</code>	cor	Cor de fundo da lista suspensa.

Opcoes comuns (compartilhadas com a maioria dos widgets)

Propriedade / Metodo	Tipo	Descricao
<code>master</code>	widget	Widget pai onde o componente sera colocado.
<code>width</code> / <code>height</code>	int	Largura e altura em pixels.
<code>corner_radius</code>	int	Raio dos cantos arredondados (visual moderno).
<code>fg_color</code>	cor/tupla	Cor de preenchimento. Aceita ('claro','escuro') por modo de tema.
<code>bg_color</code>	cor	Cor de fundo atras do widget.
<code>border_width</code>	int	Espessura da borda.
<code>border_color</code>	cor	Cor da borda.
<code>text_color</code>	cor	Cor do texto.
<code>font</code>	CTkFont/tupla	Fonte usada no texto.
<code>hover_color</code>	cor	Cor exibida ao passar o mouse (botoes/itens).
<code>state</code>	str	'normal' ou 'disabled'.

Exemplos de codigo

Exemplo 1: OptionMenu de idioma

```
import customtkinter as ctk

app = ctk.CTk(); app.geometry('300x160')
ctk.CTkOptionMenu(app, values=['Portugues', 'Ingles', 'Espanhol'],
                  command=lambda v: print(v)).pack(pady=40)
ctk.CTkComboBox(app, values=['10', '20', '30']).pack(pady=10)
app.mainloop()
```

O OptionMenu so permite escolher da lista; o ComboBox tambem aceita digitacao.

Tela de exemplo: A tela mostra um menu de idiomas e uma caixa de combinacao editavel.

CTkTabview (Abas Modernas)

Importacao: import customtkinter as ctk

O CTkTabview cria uma interface de abas moderna; cada aba e criada com add e acessada com tab(nome) para receber widgets.

Propriedades e metodos principais

Propriedade / Metodo	Tipo	Descricao
<code>add(nome)</code>	metodo	Cria uma nova aba e retorna seu frame.
<code>tab(nome)</code>	metodo	Retorna o frame de uma aba existente.
<code>set(nome)</code>	metodo	Seleciona uma aba.
<code>get()</code>	metodo	Retorna o nome da aba ativa.

Opcoes comuns (compartilhadas com a maioria dos widgets)

Propriedade / Metodo	Tipo	Descricao
<code>master</code>	widget	Widget pai onde o componente sera colocado.
<code>width / height</code>	int	Largura e altura em pixels.
<code>corner_radius</code>	int	Raio dos cantos arredondados (visual moderno).
<code>fg_color</code>	cor/tupla	Cor de preenchimento. Aceita ('claro','escuro') por modo de tema.
<code>bg_color</code>	cor	Cor de fundo atras do widget.
<code>border_width</code>	int	Espessura da borda.
<code>border_color</code>	cor	Cor da borda.
<code>text_color</code>	cor	Cor do texto.
<code>font</code>	CTkFont/tupla	Fonte usada no texto.
<code>hover_color</code>	cor	Cor exibida ao passar o mouse (botoes/itens).
<code>state</code>	str	'normal' ou 'disabled'.

Exemplos de codigo

Exemplo 1: Abas modernas

```
import customtkinter as ctk

app = ctk.CTk(); app.geometry('360x240')
tabs = ctk.CTkTabview(app)
tabs.pack(padx=20, pady=20, fill='both', expand=True)
tabs.add('Perfil'); tabs.add('Config')
ctk.CTkLabel(tabs.tab('Perfil'), text='Dados do perfil').pack(pady=20)
ctk.CTkLabel(tabs.tab('Config'), text='Configuracoes').pack(pady=20)
app.mainloop()
```

add cria abas e tab(nome) fornece o frame para adicionar widgets em cada uma.

Tela de exemplo: A tela mostra abas modernas Perfil e Config com conteudos distintos.

CTkScrollableFrame (Quadro Rolavel)

Importacao: import customtkinter as ctk

O CtkScrollableFrame e um container com rolagem automatica, ideal para listas longas de widgets que nao cabem na tela.

Propriedades e metodos principais

Propriedade / Metodo	Tipo	Descricao
<code>label_text</code>	str	Titulo opcional do quadro.
<code>orientation</code>	str	'vertical' ou 'horizontal'.
<code>fg_color</code>	cor	Cor de fundo.

Opcoes comuns (compartilhadas com a maioria dos widgets)

Propriedade / Metodo	Tipo	Descricao
<code>master</code>	widget	Widget pai onde o componente sera colocado.
<code>width / height</code>	int	Largura e altura em pixels.
<code>corner_radius</code>	int	Raio dos cantos arredondados (visual moderno).
<code>fg_color</code>	cor/tupla	Cor de preenchimento. Aceita ('claro','escuro') por modo de tema.
<code>bg_color</code>	cor	Cor de fundo atras do widget.
<code>border_width</code>	int	Espessura da borda.
<code>border_color</code>	cor	Cor da borda.
<code>text_color</code>	cor	Cor do texto.
<code>font</code>	CTkFont/tupla	Fonte usada no texto.
<code>hover_color</code>	cor	Cor exibida ao passar o mouse (botoes/itens).
<code>state</code>	str	'normal' ou 'disabled'.

Exemplos de codigo

Exemplo 1: Lista rolavel

```
import customtkinter as ctk

app = ctk.CTk(); app.geometry('300x300')
frame = ctk.CTkScrollableFrame(app, label_text='Itens')
frame.pack(fill='both', expand=True, padx=10, pady=10)
for i in range(30):
    ctk.CTkButton(frame, text=f'Item {i+1}').pack(pady=3)
app.mainloop()
```

Mesmo com 30 botoes, o quadro rola automaticamente sem ocupar toda a tela.

Tela de exemplo: A tela mostra uma lista rolavel com dezenas de botoes empilhados.

CTkTextbox e CTkSegmentedButton

Importacao: import customtkinter as ctk

O CTkTextbox e a area de texto multilinha moderna; o CTkSegmentedButton apresenta um conjunto de botoes segmentados para escolha rapida entre opcoes.

Propriedades e metodos principais

Propriedade / Metodo	Tipo	Descricao
<code>insert/get/delete</code>	metodo	Manipulam o texto do CTkTextbox.
<code>values</code>	lista	Segmentos do CTkSegmentedButton.
<code>command</code>	funcao	Funcao chamada ao escolher um segmento.
<code>set(v)</code>	metodo	Define o segmento selecionado.

Opcoes comuns (compartilhadas com a maioria dos widgets)

Propriedade / Metodo	Tipo	Descricao
<code>master</code>	widget	Widget pai onde o componente sera colocado.
<code>width / height</code>	int	Largura e altura em pixels.
<code>corner_radius</code>	int	Raio dos cantos arredondados (visual moderno).
<code>fg_color</code>	cor/tupla	Cor de preenchimento. Aceita ('claro','escuro') por modo de tema.
<code>bg_color</code>	cor	Cor de fundo atras do widget.
<code>border_width</code>	int	Espessura da borda.
<code>border_color</code>	cor	Cor da borda.
<code>text_color</code>	cor	Cor do texto.
<code>font</code>	CTkFont/tupla	Fonte usada no texto.
<code>hover_color</code>	cor	Cor exibida ao passar o mouse (botoes/itens).
<code>state</code>	str	'normal' ou 'disabled'.

Exemplos de codigo

Exemplo 1: Editor e segmentos

```
import customtkinter as ctk

app = ctk.CTk(); app.geometry('360x300')
seg = ctk.CTkSegmentedButton(app, values=['Dia', 'Semana', 'Mes'],
                             command=lambda v: print(v))

seg.set('Dia')
seg.pack(pady=15)
tb = ctk.CTkTextbox(app, width=300, height=150)
tb.pack(pady=10)
tb.insert('0.0', 'Escreva suas anotacoes aqui...')
app.mainloop()
```

O botao segmentado escolhe periodos; a textbox aceita texto multilinha.

Tela de exemplo: A tela mostra um seletor segmentado (Dia/Semana/Mes) acima de um editor de texto.

Parte VII – Projetos Completos

Chegou a hora de integrar tudo o que voce aprendeu. Os projetos a seguir sao aplicacoes completas e funcionais que combinam multiplos widgets, gerenciadores de geometria, eventos e logica de programacao. Digite, execute e depois personalize cada um deles.

Calculadora Simples (Tkinter)

Uma calculadora basica que demonstra o uso de grid para organizar botoes, Entry para o visor e callbacks para processar as operacoes. Este projeto consolida o uso de gerenciador de geometria em grade e funcoes com argumentos.

Codigo completo

```
import tkinter as tk

root = tk.Tk()
root.title('Calculadora')

visor = tk.Entry(root, font=('Arial', 20), justify='right', bd=8)
visor.grid(row=0, column=0, columnspan=4, sticky='nsew')

def digitar(valor):
    visor.insert('end', valor)

def limpar():
    visor.delete(0, 'end')

def calcular():
    try:
        resultado = eval(visor.get())
        visor.delete(0, 'end')
        visor.insert('end', str(resultado))
    except Exception:
        visor.delete(0, 'end')
        visor.insert('end', 'Erro')

botoes = [
    ('7', 1, 0), ('8', 1, 1), ('9', 1, 2), ('/', 1, 3),
    ('4', 2, 0), ('5', 2, 1), ('6', 2, 2), ('*', 2, 3),
    ('1', 3, 0), ('2', 3, 1), ('3', 3, 2), ('-', 3, 3),
    ('0', 4, 0), ('.', 4, 1), ('+', 4, 2),
]

for (txt, lin, col) in botoes:
    tk.Button(root, text=txt, font=('Arial', 16),
              command=lambda t=txt: digitar(t)).grid(row=lin, column=col, sticky='nsew')

tk.Button(root, text='=', font=('Arial', 16), bg='#22c55e',
          command=calcular).grid(row=4, column=3, sticky='nsew')
tk.Button(root, text='C', font=('Arial', 16), bg='#ef4444',
          command=limpar).grid(row=5, column=0, columnspan=4, sticky='nsew')

for i in range(6):
    root.rowconfigure(i, weight=1)
for i in range(4):
    root.columnconfigure(i, weight=1)

root.mainloop()
```

O grid posiciona o visor e os botoes em linhas e colunas. rowconfigure e columnconfigure com weight permitem que a janela seja redimensionavel. A funcao eval interpreta a expressao digitada (em producao, prefira um parser seguro).

Lista de Tarefas (To-Do List)

Aplicativo de tarefas que usa Entry, Listbox e Button para adicionar, marcar e remover itens. Demonstra leitura de selecao e atualizacao dinamica da lista.

Codigo completo

```
import tkinter as tk

root = tk.Tk()
root.title('Minhas Tarefas')
root.geometry('360x420')

entrada = tk.Entry(root, font=('Arial', 12))
entrada.pack(fill='x', padx=10, pady=10)

lista = tk.Listbox(root, font=('Arial', 12), height=12)
lista.pack(fill='both', expand=True, padx=10)

def adicionar():
    texto = entrada.get().strip()
    if texto:
        lista.insert('end', texto)
        entrada.delete(0, 'end')

def concluir():
    sel = lista.curselection()
    if sel:
        i = sel[0]
        texto = lista.get(i)
        if not texto.startswith('[OK] '):
            lista.delete(i)
            lista.insert(i, '[OK] ' + texto)

def remover():
    sel = lista.curselection()
    if sel:
        lista.delete(sel[0])

barra = tk.Frame(root)
barra.pack(fill='x', padx=10, pady=10)
tk.Button(barra, text='Adicionar', command=adicionar).pack(side='left', expand=True, fill='x')
tk.Button(barra, text='Concluir', command=concluir).pack(side='left', expand=True, fill='x')
tk.Button(barra, text='Remover', command=remover).pack(side='left', expand=True, fill='x')

entrada.bind('<Return>', lambda e: adicionar())
root.mainloop()
```

O Entry captura a nova tarefa, a Listbox armazena os itens e os botoes manipulam a selecao atual. O bind em <Return> permite adicionar pressionando Enter, melhorando a usabilidade.

Tela de Login Moderna (CustomTkinter)

Tela de autenticação com visual moderno usando CtkEntry, CtkButton e CtkLabel. Demonstra layout centralizado, placeholders e validação simples de credenciais.

Código completo

```
import customtkinter as ctk
from tkinter import messagebox

ctk.set_appearance_mode('dark')
ctk.set_default_color_theme('blue')

app = ctk.CTk()
app.title('Login')
app.geometry('400x480')

card = ctk.CTkFrame(app, corner_radius=20)
card.pack(expand=True, padx=40, pady=40, fill='both')

ctk.CTkLabel(card, text='Bem-vindo',
              font=ctk.CTkFont(size=26, weight='bold')).pack(pady=(40, 20))

usuario = ctk.CTkEntry(card, placeholder_text='Usuario', width=240, height=40)
usuario.pack(pady=10)
senha = ctk.CTkEntry(card, placeholder_text='Senha', show='*', width=240, height=40)
senha.pack(pady=10)

def entrar():
    if usuario.get() == 'admin' and senha.get() == '1234':
        messagebox.showinfo('Sucesso', 'Login realizado!')
    else:
        messagebox.showerror('Erro', 'Credenciais invalidas.')

ctk.CTkButton(card, text='Entrar', width=240, height=40,
               command=entrar).pack(pady=20)
ctk.CTkButton(card, text='Criar conta', width=240, height=40,
               fg_color='transparent', border_width=1).pack()

app.mainloop()
```

Um CtkFrame com cantos arredondados agrupa os campos como um cartão. O botão secundário usa `fg_color='transparent'` com borda para um estilo 'outline'. A validação compara as credenciais e exibe mensagens.

Bloco de Notas com Salvamento (Tkinter)

Editor de texto que usa o widget Text, menus e os dialogos de arquivo para abrir e salvar documentos. Demonstra integracao com filedialog e manipulacao de arquivos.

Codigo completo

```
import tkinter as tk
from tkinter import filedialog, messagebox

root = tk.Tk()
root.title('Bloco de Notas')
root.geometry('600x450')

area = tk.Text(root, font=('Consolas', 12), wrap='word', undo=True)
area.pack(fill='both', expand=True)

def abrir():
    caminho = filedialog.askopenfilename(filetypes=[('Texto', '*.txt')])
    if caminho:
        with open(caminho, 'r', encoding='utf-8') as f:
            area.delete('1.0', 'end')
            area.insert('1.0', f.read())

def salvar():
    caminho = filedialog.asksaveasfilename(defaultextension='.txt',
                                           filetypes=[('Texto', '*.txt')])
    if caminho:
        with open(caminho, 'w', encoding='utf-8') as f:
            f.write(area.get('1.0', 'end-1c'))
        messagebox.showinfo('Salvo', 'Arquivo salvo com sucesso!')

barra = tk.Menu(root)
arquivo = tk.Menu(barra, tearoff=0)
arquivo.add_command(label='Abrir', command=abrir)
arquivo.add_command(label='Salvar', command=salvar)
arquivo.add_separator()
arquivo.add_command(label='Sair', command=root.destroy)
barra.add_cascade(label='Arquivo', menu=arquivo)
root.config(menu=barra)

root.mainloop()
```

filedialog abre janelas nativas para escolher arquivos. O Text com undo=True habilita desfazer (Ctrl+Z). O conteudo e salvo com codificacao UTF-8 para preservar acentos.

Conversor de Temperatura (Tkinter + ttk)

Conversor entre Celsius, Fahrenheit e Kelvin usando Combobox para selecionar as unidades e Entry para o valor. Demonstra leitura de selecao e calculos.

Codigo completo

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()
root.title('Conversor de Temperatura')
root.geometry('320x240')

valor = tk.Entry(root, font=('Arial', 14), justify='center')
valor.pack(pady=15)

origem = ttk.Combobox(root, values=['Celsius', 'Fahrenheit', 'Kelvin'],
                      state='readonly')
origem.current(0); origem.pack(pady=5)
destino = ttk.Combobox(root, values=['Celsius', 'Fahrenheit', 'Kelvin'],
                      state='readonly')
destino.current(1); destino.pack(pady=5)

resultado = tk.Label(root, text='', font=('Arial', 14, 'bold'))
resultado.pack(pady=15)

def para_celsius(v, u):
    return v if u == 'Celsius' else (v - 32) * 5/9 if u == 'Fahrenheit' else v - 273.15

def de_celsius(c, u):
    return c if u == 'Celsius' else c * 9/5 + 32 if u == 'Fahrenheit' else c + 273.15

def converter():
    try:
        c = para_celsius(float(valor.get()), origem.get())
        r = de_celsius(c, destino.get())
        resultado.config(text=f'{r:.2f} {destino.get()}')
    except ValueError:
        resultado.config(text='Valor invalido')

tk.Button(root, text='Converter', command=converter).pack()
root.mainloop()
```

A logica converte qualquer unidade para Celsius e de Celsius para a unidade de destino, simplificando os calculos. Combobox readonly garante selecoes validas.

Dashboard com Barra Lateral (CustomTkinter)

Layout de painel administrativo com barra lateral de navegacao e area de conteudo que troca conforme o botao clicado. Demonstra organizacao em frames e troca de telas.

Codigo completo

```
import customtkinter as ctk

ctk.set_appearance_mode('dark')
app = ctk.CTk()
app.title('Dashboard')
app.geometry('700x450')
app.grid_columnconfigure(1, weight=1)
app.grid_rowconfigure(0, weight=1)

lateral = ctk.CTkFrame(app, width=160, corner_radius=0)
lateral.grid(row=0, column=0, sticky='nsew')

conteudo = ctk.CTkFrame(app)
conteudo.grid(row=0, column=1, sticky='nsew', padx=20, pady=20)

titulo = ctk.CTkLabel(conteudo, text='Inicio',
                      font=ctk.CTkFont(size=24, weight='bold'))
titulo.pack(pady=30)

def mostrar(nome):
    titulo.configure(text=nome)

ctk.CTkLabel(lateral, text='MENU',
              font=ctk.CTkFont(size=16, weight='bold')).pack(pady=20)
for nome in ['Inicio', 'Relatorios', 'Usuarios', 'Config']:
    ctk.CTkButton(lateral, text=nome,
                  command=lambda n=nome: mostrar(n)).pack(pady=8, padx=15, fill='x')

app.mainloop()
```

O grid divide a janela em barra lateral fixa e conteudo expansivel. Cada botao da barra atualiza o titulo do conteudo, simulando a navegacao entre secoes de um painel.

Parte VIII – Exercicios Propostos

Pratique o que aprendeu resolvendo os exercicios abaixo. Tente resolver cada um sozinho antes de consultar a dica de solucao.

Exercicio 1. Crie uma janela com um Label que exhibe 'Ola Mundo' centralizado.

Dica: Use `tk.Label` com `text` e o metodo `pack(expand=True)` para centralizar.

Exercicio 2. Faca um botao que, ao ser clicado, troca o texto de um Label.

Dica: Guarde o Label em uma variavel e use `label.config(text=...)` dentro do callback.

Exercicio 3. Monte um formulario com nome, e-mail e um botao que imprime os dados.

Dica: Use tres `Entry` e leia cada um com `get()` dentro da funcao do botao.

Exercicio 4. Crie um contador com botoes de + e - que atualizam um Label.

Dica: Mantenha o valor em uma variavel e atualize o Label apos cada clique.

Exercicio 5. Implemente uma lista onde se adiciona itens digitados em um `Entry`.

Dica: Combine `Entry` e `Listbox`; no botao, faca `listbox.insert('end', entry.get())`.

Exercicio 6. Crie um `Checkbutton` que habilita/desabilita um botao.

Dica: No command do `Checkbutton`, use `button.config(state=...)` conforme a variavel.

Exercicio 7. Monte um grupo de `Radiobuttons` para escolher um tema de cor.

Dica: Compartilhe uma `StringVar` e aplique a cor escolhida ao fundo da janela.

Exercicio 8. Faca um `Scale` que altera o tamanho da fonte de um Label.

Dica: No command do `Scale`, use `label.config(font=('Arial', int(valor)))`.

Exercicio 9. Crie uma calculadora de IMC com dois `Entry` e um Label de resultado.

Dica: Leia peso e altura, calcule $\text{peso}/(\text{altura}^2)$ e exiba com formatacao.

Exercicio 10. Construa uma tela com abas usando `ttk.Notebook` contendo 3 paginas.

Dica: Crie 3 frames e adicione cada um com `nb.add(frame, text=...)`.

Exercicio 11. Implemente uma barra de progresso que avanca com um botao.

Dica: Use `ttk.Progressbar` e incremente `pb['value']` no callback.

Exercicio 12. Refaca a tela de login usando `CustomTkinter` com tema escuro.

Dica: Use `CTkEntry` com `placeholder_text` e valide as credenciais no botao.

Parte IX – Referencias Rápidas

Constantes do Tkinter

Categoria	Valores	Descrição
Side (pack)	LEFT, RIGHT, TOP, BOTTOM	Define o lado onde o widget é ancorado no pack.
Fill (pack)	X, Y, BOTH, NONE	Faz o widget preencher o espaço horizontal/vertical.
Anchor	N, S, E, W, NE, NW, SE, SW, CENTER	Posiciona o conteúdo dentro do espaço disponível.
Sticky (grid)	'n', 's', 'e', 'w' e combinações	Estica o widget na célula do grid (ex.: 'nsew').
Relief	FLAT, RAISED, SUNKEN, GROOVE, RIDGE, SOLID	Estilo visual da borda do widget.
State	NORMAL, DISABLED, ACTIVE, 'readonly'	Estado de interação do widget.
Justify	LEFT, RIGHT, CENTER	Alinhamento de texto com várias linhas.
Orient	HORIZONTAL, VERTICAL	Orientação de Scale, Scrollbar e Progressbar.
Selectmode	SINGLE, BROWSE, MULTIPLE, EXTENDED	Modo de seleção da Listbox.
Wrap (Text)	WORD, CHAR, NONE	Modo de quebra de linha no widget Text.
Compound	LEFT, RIGHT, TOP, BOTTOM, CENTER	Combina texto e imagem em um widget.

Cursors disponíveis

Use a opção cursor para mudar o formato do ponteiro sobre um widget. Exemplos de valores: arrow, hand2, watch, cross, fleur, xterm, circle, plus, spider, pirate, heart, star, trek, sizing, sb_h_double_arrow, sb_v_double_arrow, exchange, question_arrow.

Boas práticas gerais

- Separe a interface (widgets) da lógica de negócio em funções ou classes.
- Use variáveis de controle (StringVar, IntVar, BooleanVar) para sincronizar dados e widgets.
- Prefira o gerenciador grid para formulários e pack para layouts simples; nunca os misture no mesmo container.
- Defina nomes descritivos para os widgets e funções de callback.
- Para aplicações maiores, organize a interface em classes que herdam de tk.Frame ou ctk.CTkFrame.
- Evite o eval em entradas do usuário em produção; use parsers seguros.
- Sempre trate exceções ao ler arquivos ou converter valores digitados.
- Teste a interface redimensionando a janela para garantir um layout responsivo.

Erros comuns e como evitar

- Esquecer o `mainloop()`: a janela abre e fecha imediatamente.
- Chamar a funcao no command com parenteses (`command=func()`) em vez de passar a referencia (`command=func`).
- Misturar pack e grid no mesmo container, causando travamento do layout.
- Usar variaveis de loop diretamente em lambdas sem captura (use `lambda x=x: ...`).
- Nao guardar referencia de `PhotoImage`, fazendo a imagem desaparecer (coletada pelo garbage collector).
- Tentar ler um widget antes de criar a janela root.

Fim da apostila – bons estudos e bons projetos!